Reference) Stanford CS231n: Deep Learning for Computer Vision
http://cs231n.stanford.edu/

Image Classification:
CNN Backbones and Transfer Learning

Sunglok Choi, Assistant Professor, Ph.D.
Computer Science and Engineering Department, SEOULTECH
sunglok@seoultech.ac.kr | https://mint-lab.github.io/

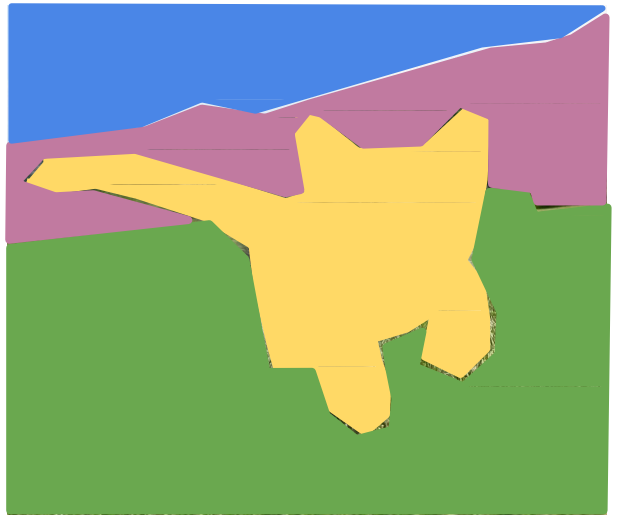
Computer Vision Tasks

Image Classification

CAT

No spatial extent

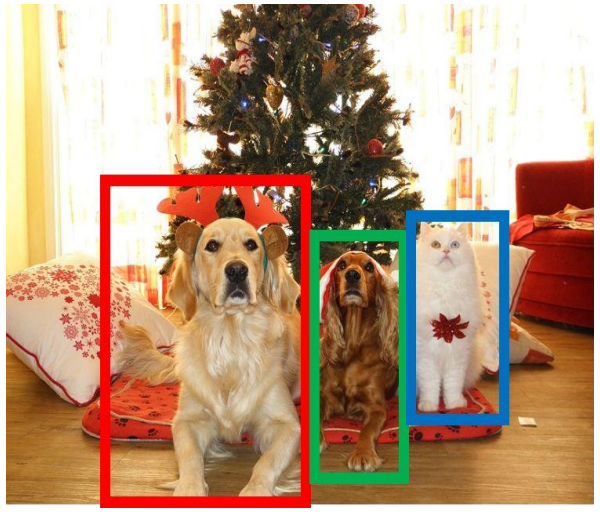
Semantic Segmentation



GRASS, CAT, TREE, SKY

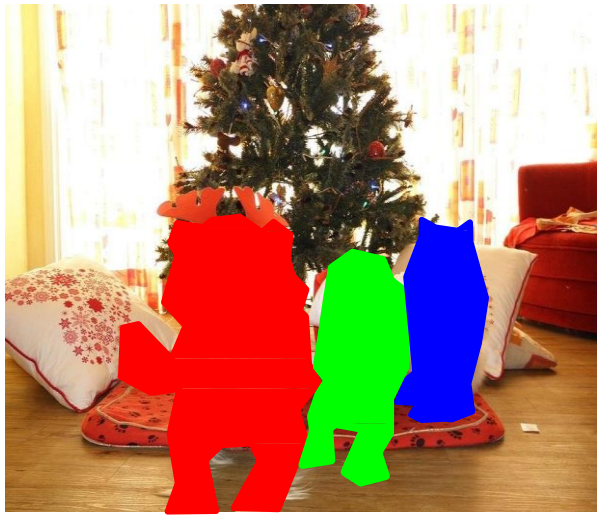
No object, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

Multiple objects

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Image Classification

- **Image classification** is a technique to **identify a label of an image** among the given label set.
  - e.g. A label set: {dog, cat, truck, plane, ...}

cat

dog

bird

deer

truck

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

- **ImageNet** is an image database organized according to the WordNet hierarchy (currently only the nouns).
- ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
  - From 2010 to 2017 (now available in Kaggle)
  - Main challenges
    - Image classification
    - Object detection (and localization)
    - …

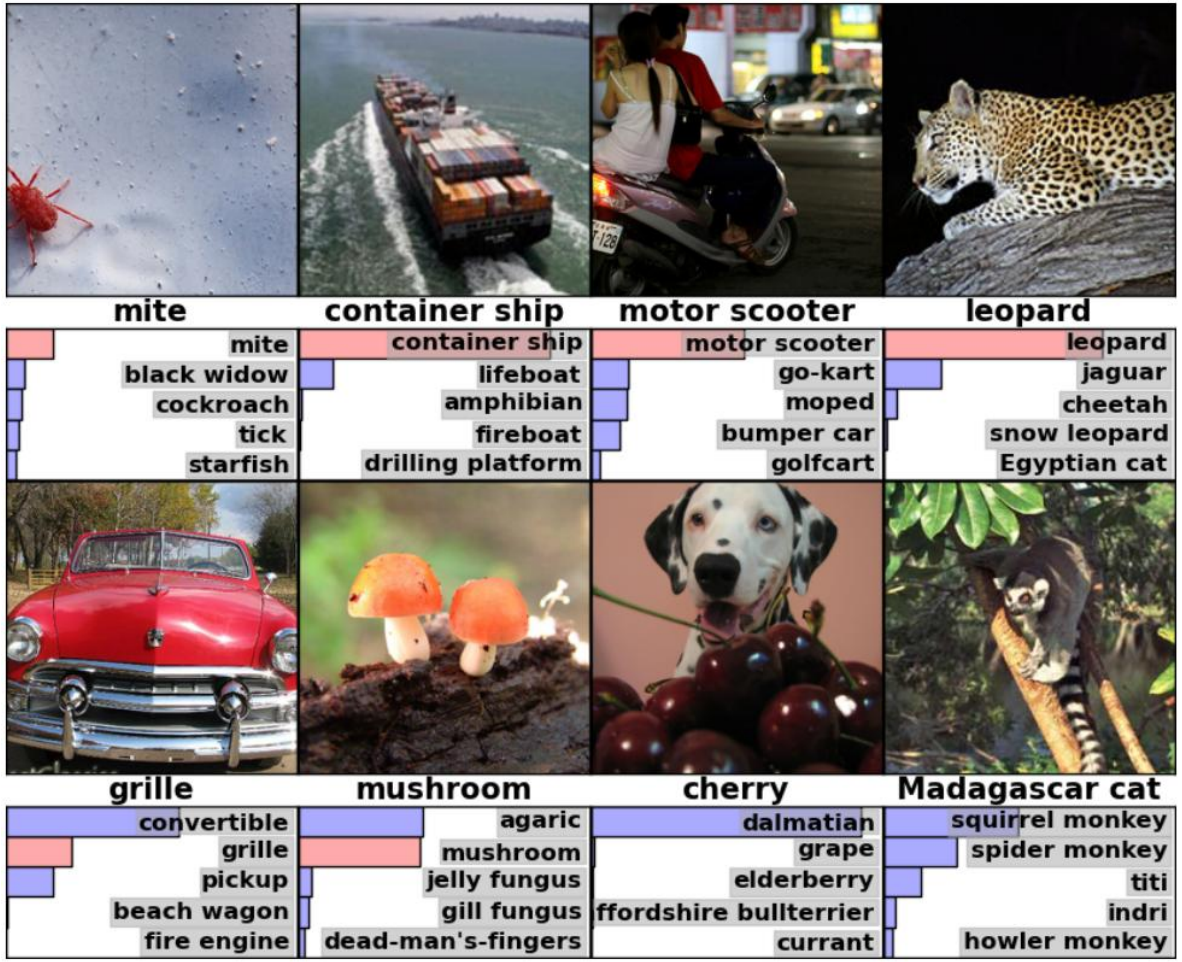
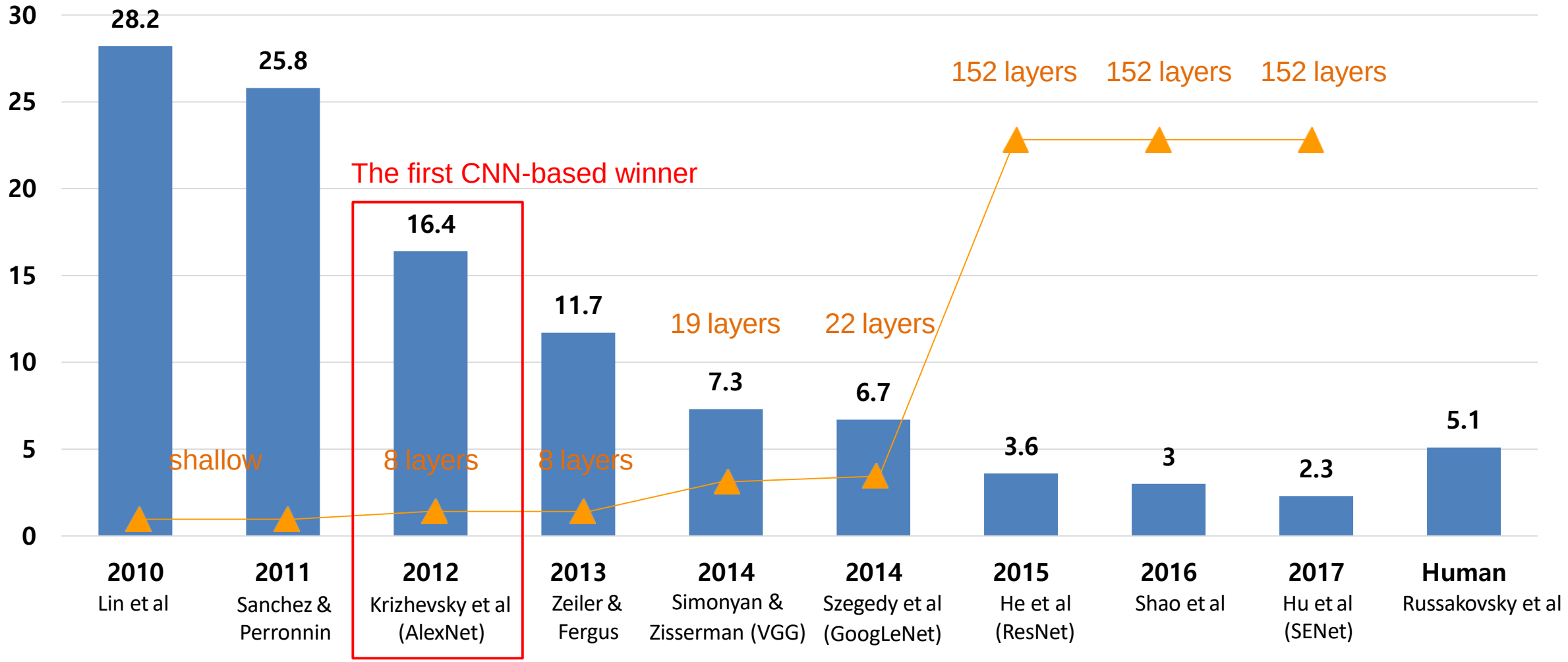


Image: Krizhevsky et al., **ImageNet Classification with Deep Convolutional Neural Networks**, NIPS, 2012 Web DOI

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

- **Top-5 errors** with their number of NN layers

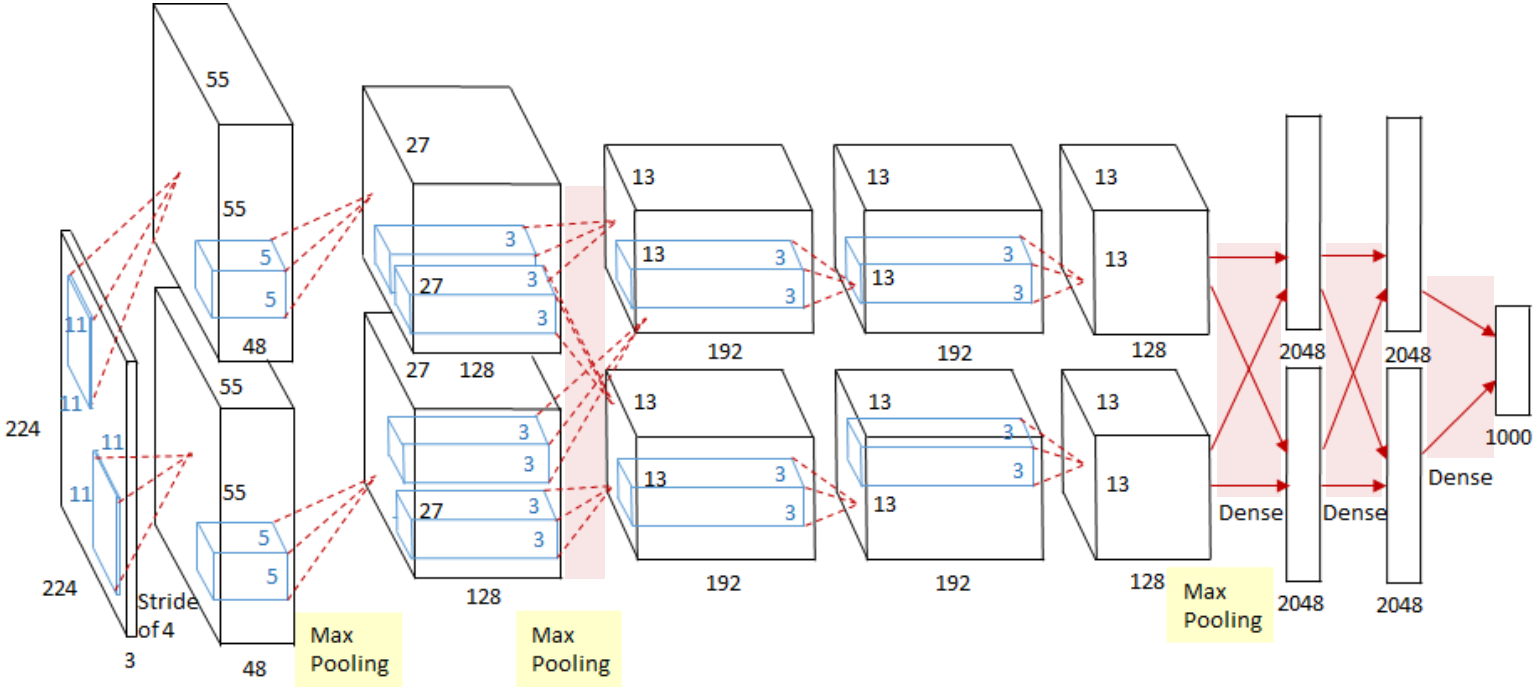


Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Image: Medium article (by Sik-Ho Tsang)

AlexNet (2012)

- The **first CNN-based winner** in ImageNet classification 2012
- Network design
  - A **8-layer** neural network (approx. 61 million parameters) with 2 x GPUs (e.g. 2 x 48 = 96 channels)
    - Historical note: AlexNet was trained on NVIDIA GTX 580 GPU with only 3 GB memory.
  - ReLU (rectified linear unit) (for faster computation and less vanishing gradients)
  - Local response normalization (for generalization; ~ batch normalization)
  - Dropout (for regularization)
- Dataset
  - ImageNet dataset
  - Data augmentation
    - Random crop (from 256 x 256)
    - Horizontal flip
    - RGB intensity transformation



Reference: Krizhevsky et al., **ImageNet Classification with Deep Convolutional Neural Networks**, NIPS, 2012 Web DOI

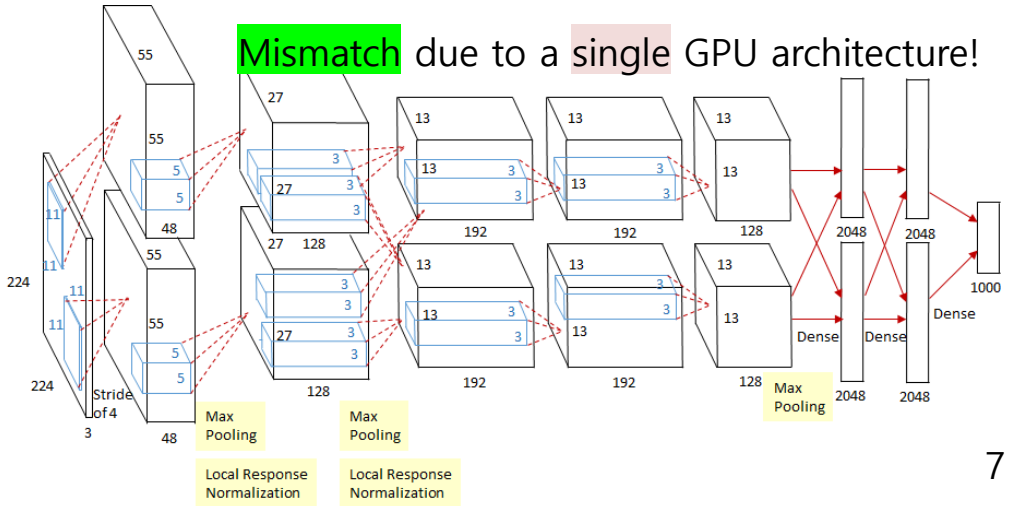
AlexNet (2012)

```python
import torchvision.models as models

model = models.alexnet(pretrained=False)
print(model)
```

- Example) **AlexNet implementation** @ PyTorch (src, doc)

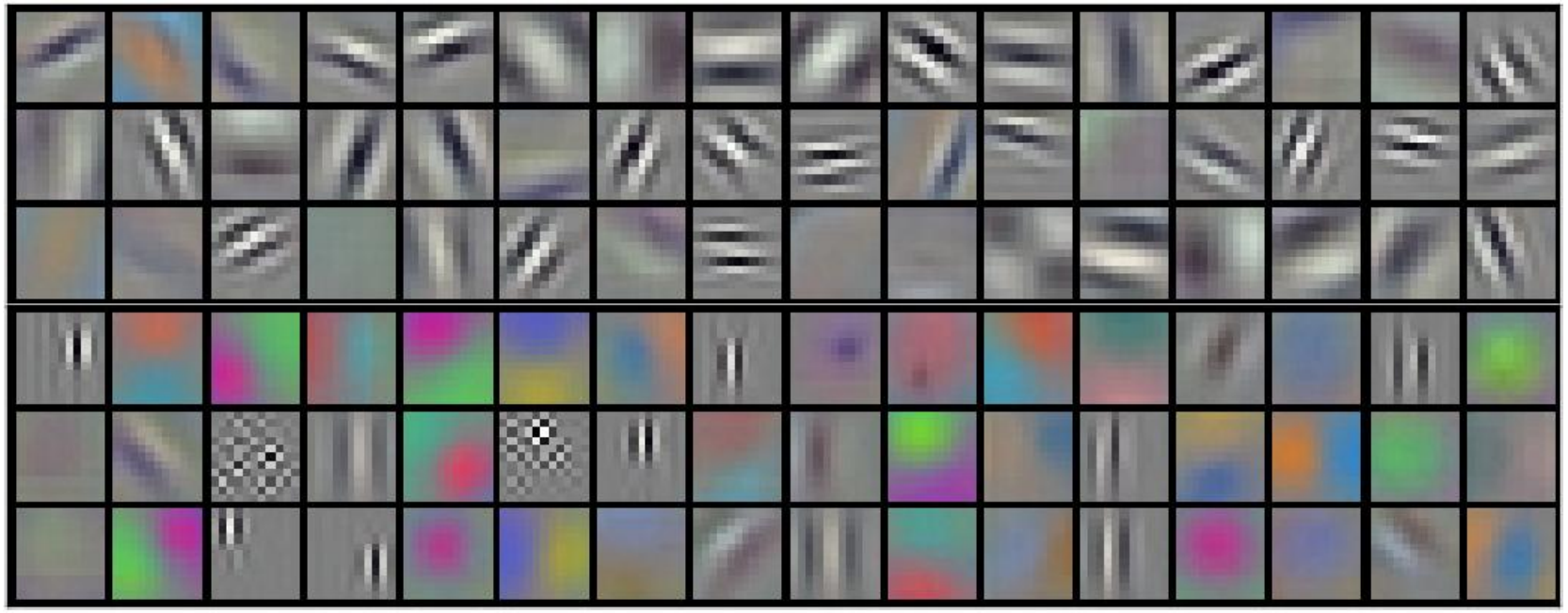
```
AlexNet(
  (features): Sequential(
    ( 0): Conv2d(  3,  64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    ( 1): ReLU(inplace=True)
    ( 2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    ( 3): Conv2d( 64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    ( 4): ReLU(inplace=True)
    ( 5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    ( 6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ( 7): ReLU(inplace=True)
    ( 8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ( 9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

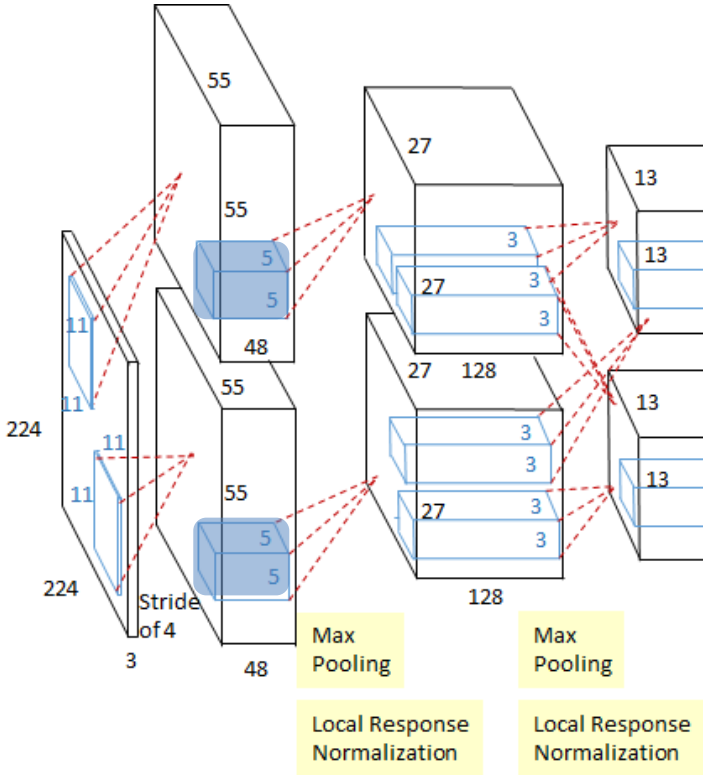
Mismatch due to a single GPU architecture!



AlexNet (2012)

- **Two important points of a convolutional layer** (in contrast to a FC layer)
  - Weight sharing
  - Local connectivity
- **Q) How does a convolution layer work?**
  - e.g. 96 convolutional kernels (11x11x3) of AlexNet





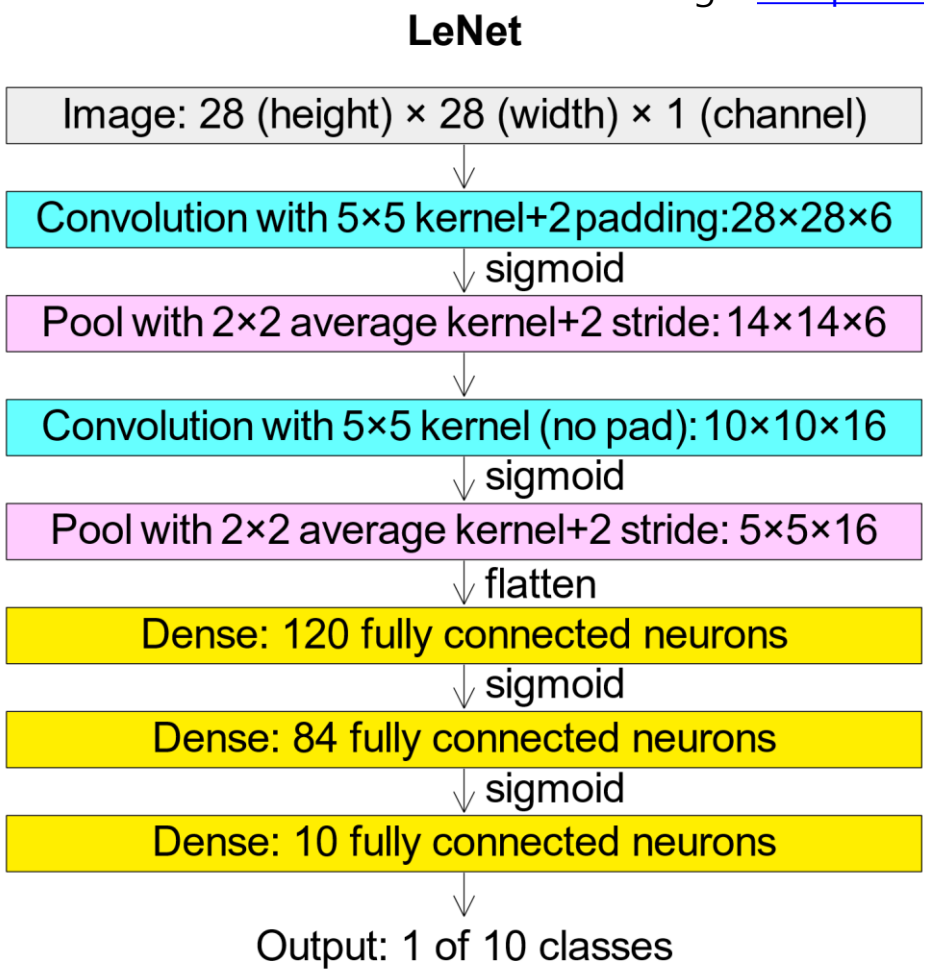
  - CNN visualization examples: ConvNetJS, ~~CNN Explainer~~

Image: Krizhevsky et al., **ImageNet Classification with Deep Convolutional Neural Networks**, NIPS, 2012 Web DOI

LeNet-5 (1998)

- The **first** CNN network proposed by LeCun et al. in 1998
  - Its original form was proposed in 1989.
- Network design
  - Convolutional layer x 2
    - Convolution + Sigmoid activation function + Average pooling
  - FC layer x 2 (for classification)
- Loss function
  - Mean squared error (MSE) with more terms
- Datasets
  - MINIST dataset

Image: Wikipedia



Reference: LeCun et al., **Gradient-based Learning Applied to Document Recognition**, Proceedings of the IEEE, vol. 86, no. 11, 1998 DOI

Image: Wikipedia

LeNet-5 (1998)

LeNet

Image: 28 (height) × 28 (width) × 1 (channel)

↓

Convolution with 5×5 kernel+2 padding: 28×28×6

↓ sigmoid

Pool with 2×2 average kernel+2 stride: 14×14×6

↓

Convolution with 5×5 kernel (no pad): 10×10×16

↓ sigmoid

Pool with 2×2 average kernel+2 stride: 5×5×16

↓ flatten

Dense: 120 fully connected neurons

↓ sigmoid

Dense: 84 fully connected neurons

↓ sigmoid

Dense: 10 fully connected neurons

↓

Output: 1 of 10 classes

AlexNet

Image: 224 (height) × 224 (width) × 3 (channels)

↓

Convolution with 11×11 kernel+4 stride: 54×54×96

↓ ReLu

Pool with 3×3 max. kernel+2 stride: 26×26×96

↓

Convolution with 5×5 kernel+2 pad: 26×26×256

↓ ReLu

Pool with 3×3 max. kernel+2 stride: 12×12×256

↓

Convolution with 3×3 kernel+1 pad: 12×12×384

↓ ReLu

Convolution with 3×3 kernel+1 pad: 12×12×384

↓ ReLu

Convolution with 3×3 kernel+1 pad: 12×12×256

↓ ReLu

Pool with 3×3 max. kernel+2 stride: 5×5×256

↓ flatten

Dense: 4096 fully connected neurons

↓ ReLu, dropout p=0.5

Dense: 4096 fully connected neurons

↓ ReLu, dropout p=0.5

Dense: 1000 fully connected neurons

↓

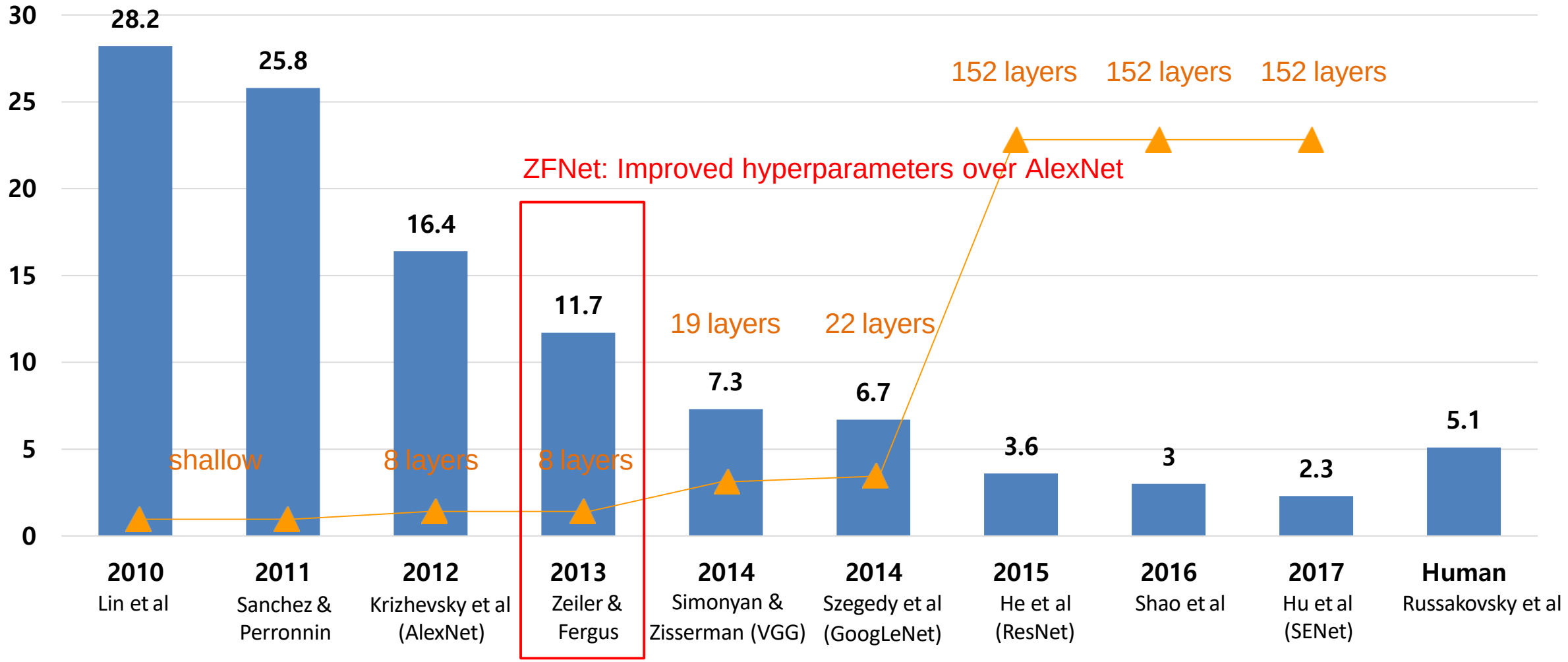
Output: 1 of 1000 classes

Training 10^7 pixels → 10^{14} pixels

1. Big data
2. Faster (massively parallel) hardware
3. Bigger models + New techniques

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

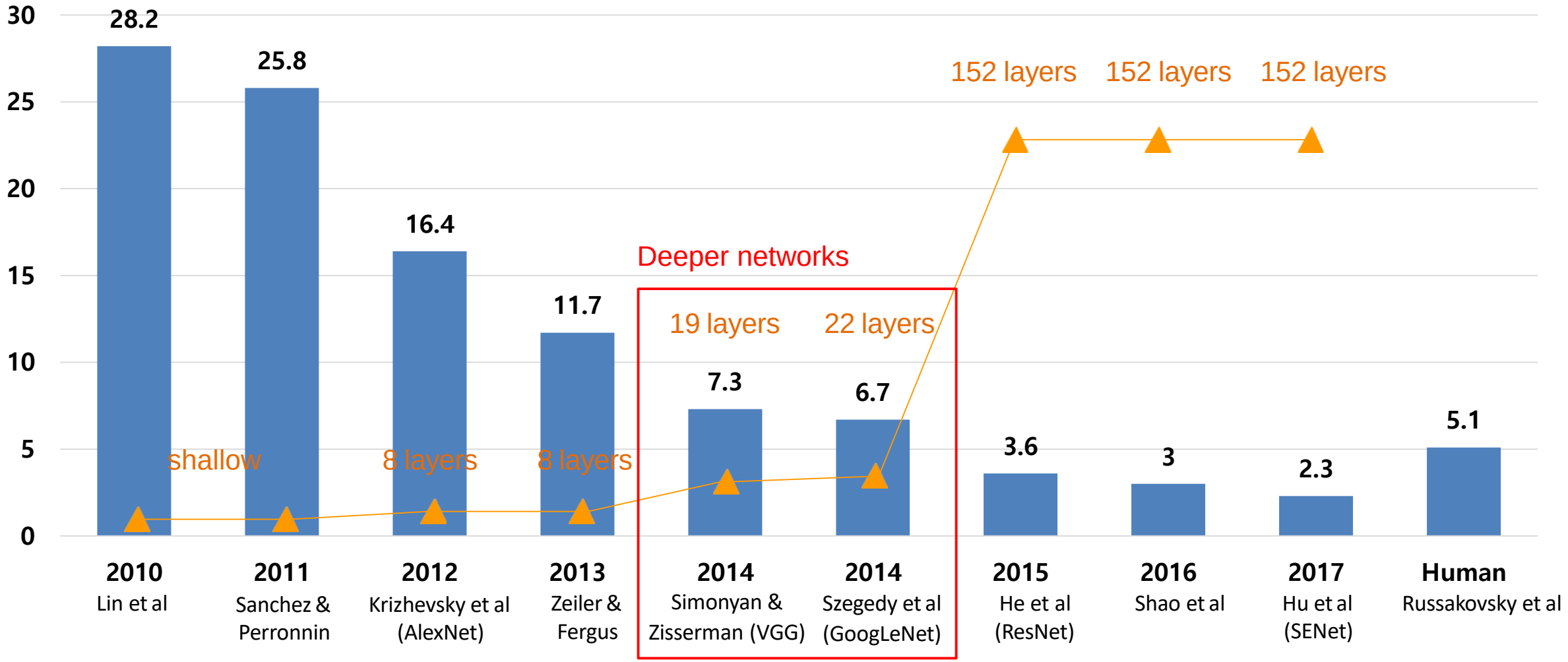
- **Top-5 errors** with their number of NN layers



Slide: [Stanford CS231n: Deep Learning for Computer Vision]

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

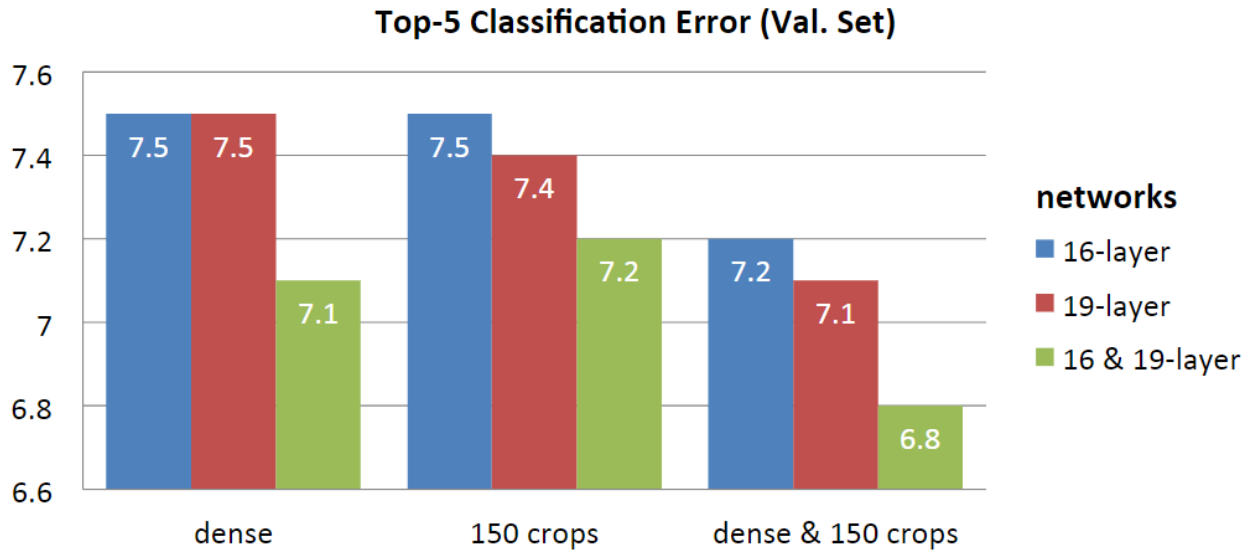
- **Top-5 errors** with their number of NN layers



Slide: [Stanford CS231n: Deep Learning for Computer Vision]

VGGNet (2014)

- The 2nd place in ImageNet classification 2014
  - The 1st place in the object localization 2014
- Network design: **"Small filter but deeper network"**
  - A stack of 3x3 convolutional layers
  - Increasing depth from 11 to 19 layers
  - Use ensembles for best results (16 & 19-layer)
- Data augmentation
  - 1 image → 150 images (scale, crop, flip, …)



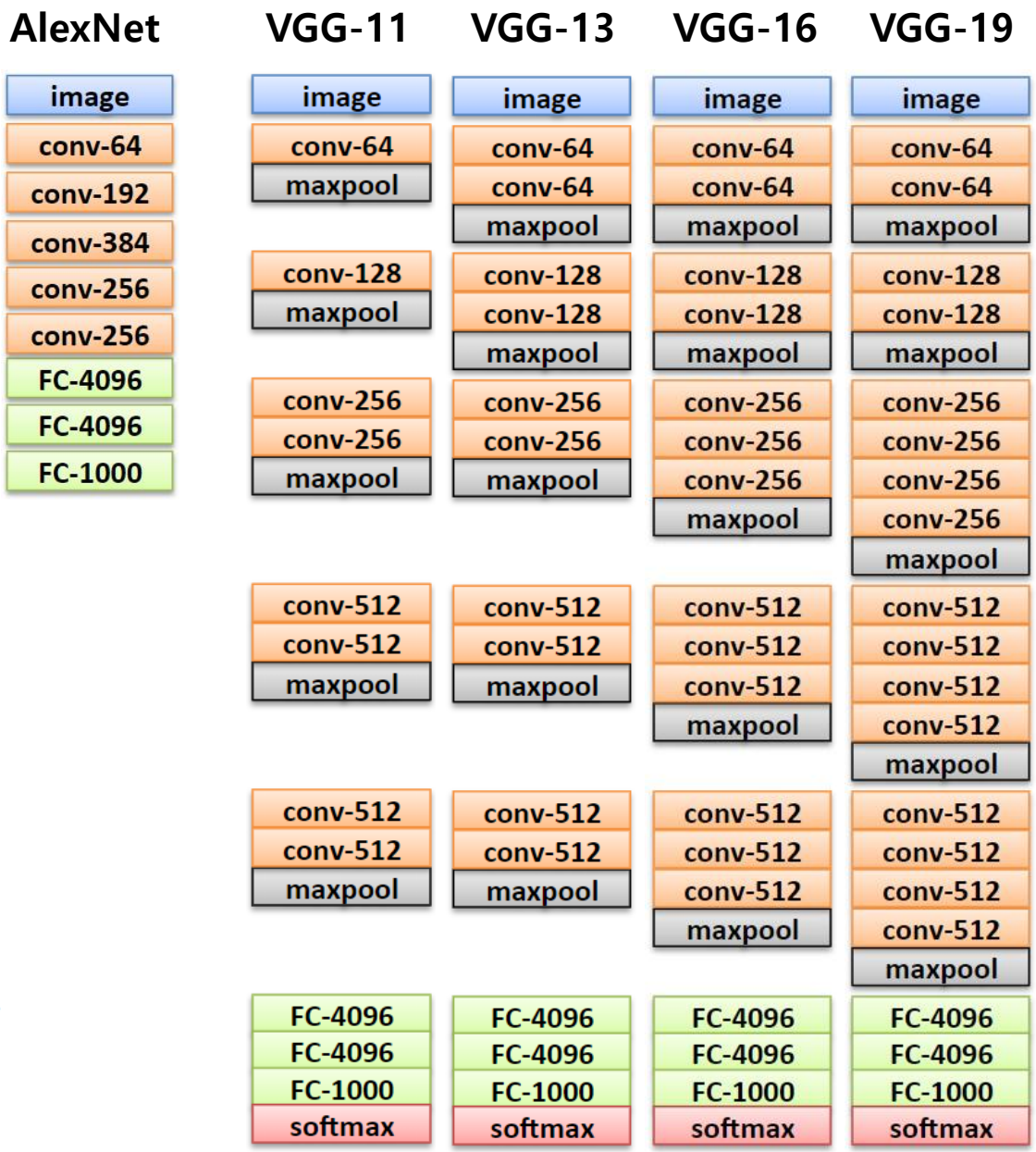
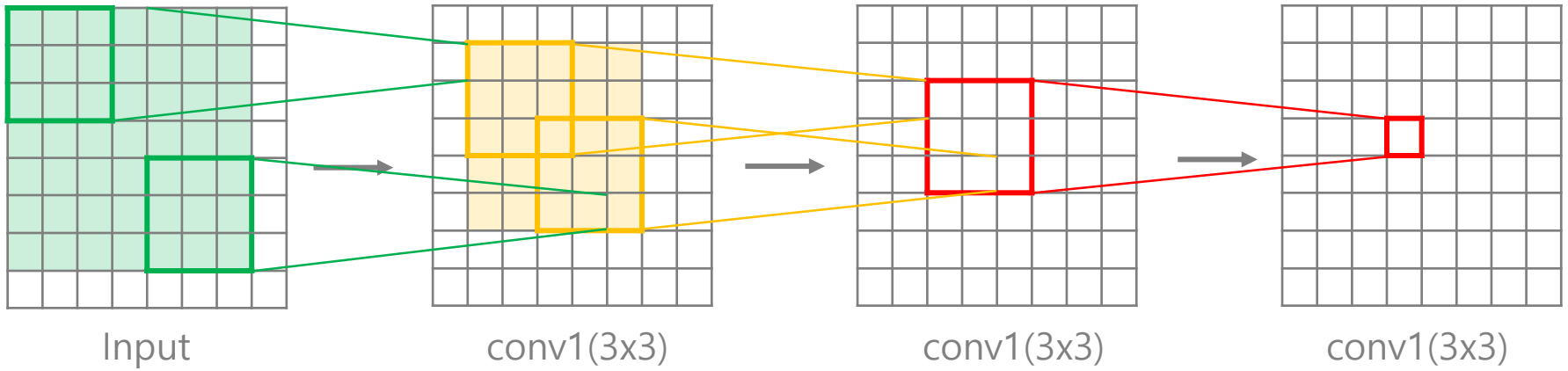


Image: Simonyan and Zisserman, **Very Deep Convolutional Networks for Large-Scale Image Recognition**, ICLR, 2015 arXiv Slides

VGGNet (2014)

- **Q) How does stacks of 3x3 filters work?**
  - A stack of 3x3 layers can have a large receptive field if they are deeper.
    - e.g. Two 3x3 layers - 5x5 receptive field
    - e.g. Three 3x3 layers - 7x7 receptive field



  - More ReLU non-linearities
  - Less parameters than a single layer
  - Design simplicity

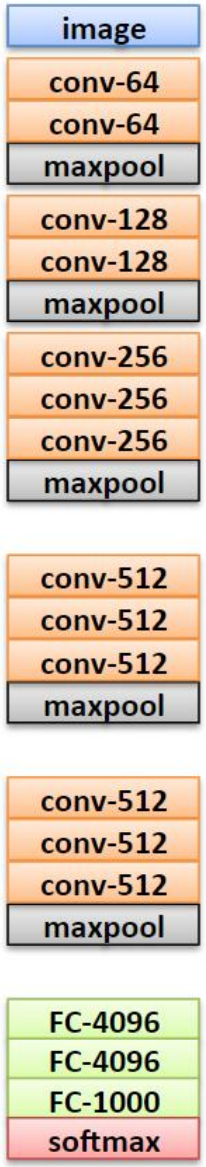
Image: [Stanford CS231n: Deep Learning for Computer Vision]

VGGNet (2014)

```python
import torchvision.models as models

model = models.vgg16(pretrained=False)
print(model)
```

- Example) **VGG-16 implementation** @ PyTorch (src, doc)

```
VGG(
  (features): Sequential(
    ( 0): Conv2d(  3,  64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ( 1): ReLU(inplace=True)
    ( 2): Conv2d( 64,  64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ( 3): ReLU(inplace=True)
    ( 4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    ( 5): Conv2d( 64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ( 6): ReLU(inplace=True)
    ( 7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ( 8): ReLU(inplace=True)
    ( 9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (13): ReLU(inplace=True)
    (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (15): ReLU(inplace=True)
    (16): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (17): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (18): ReLU(inplace=True)
    (19): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (20): ReLU(inplace=True)
    (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (22): ReLU(inplace=True)
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
```



VGGNet (2014)

- Example) **VGG-16**

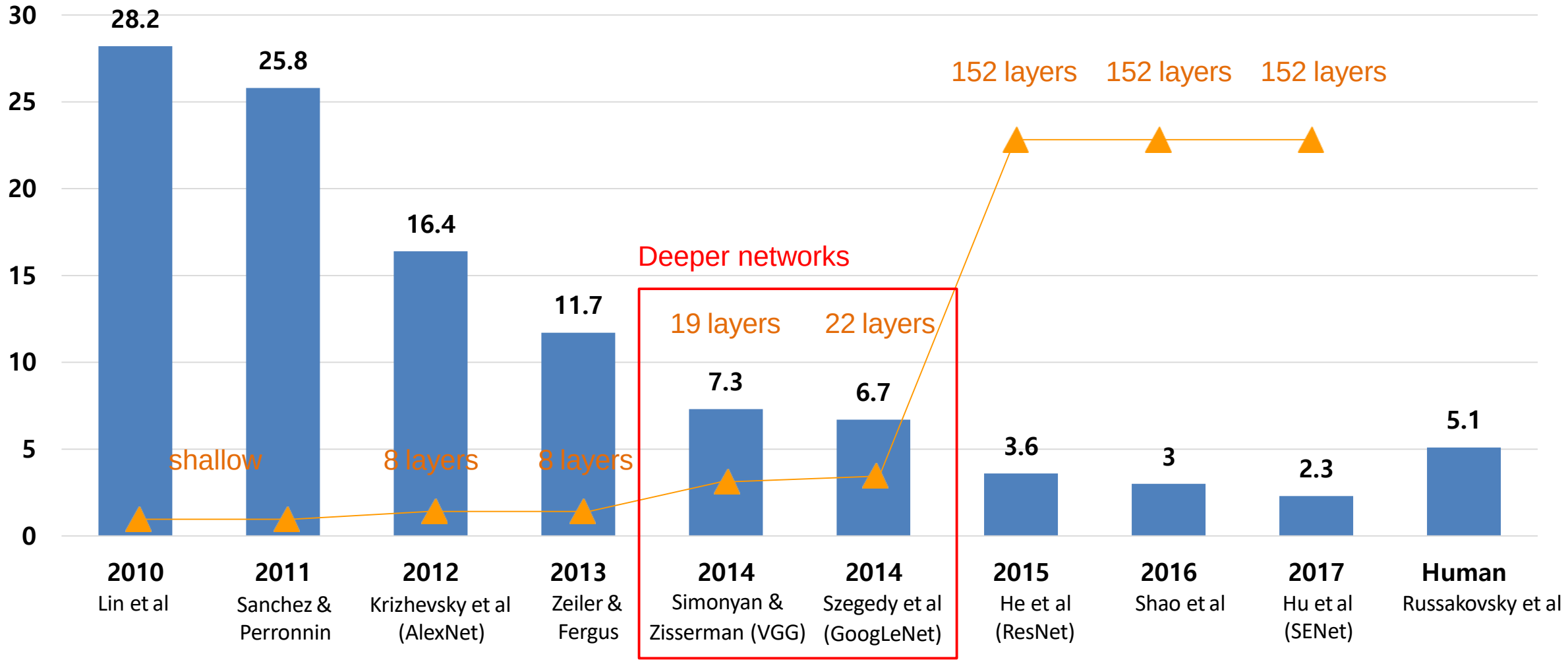
Layers	Output Shapes	Parameters	Memory (Activation)
Image	3×224×224	0	~ 0.57 MB
conv-64	64×224×224	1,792	~ 12.3 MB
conv-64	64×224×224	36,928	~ 12.3 MB
maxpool	64×112×112	0	~ 3.1 MB
conv-128	128×112×112	73,856	~ 6.1 MB
conv-128	128×112×112	147,584	~ 6.1 MB
maxpool	128×56×56	0	~ 1.2 MB
conv-256	256×56×56	295,168	~ 3.2 MB
conv-256	256×56×56	590,080	~ 3.2 MB
conv-256	256×56×56	590,080	~ 3.2 MB
maxpool	256×28×28	0	~ 0.56 MB
conv-512	512×28×28	1,180,160	~ 1.6 MB
conv-512	512×28×28	2,359,808	~ 1.6 MB
conv-512	512×28×28	2,359,808	~ 1.6 MB
maxpool	512×14×14	0	~ 0.39 MB
conv-512	512×14×14	2,359,808	~ 0.39 MB
conv-512	512×14×14	2,359,808	~ 0.39 MB
conv-512	512×14×14	2,359,808	~ 0.39 MB
maxpool	512×7×7	0	~ 0.10 MB
flatten	25088	0	~ 0.10 MB
FC-4096	4096	102,764,544	~ 0.016 MB
FC-4096	4096	16,781,312	~ 0.016 MB
FC-1000	1000	4,097,000	~ 0.004 MB
Total		**~ 138 million**	**~ 52-55 MB (per 1 image)**

Most memory is used in early conv layers.

Most parameters are in FC layers.

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

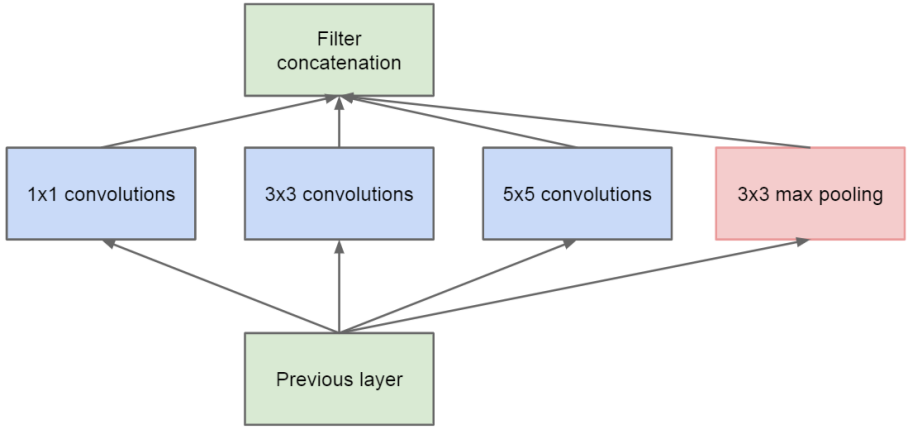
- **Top-5 errors** with their number of NN layers



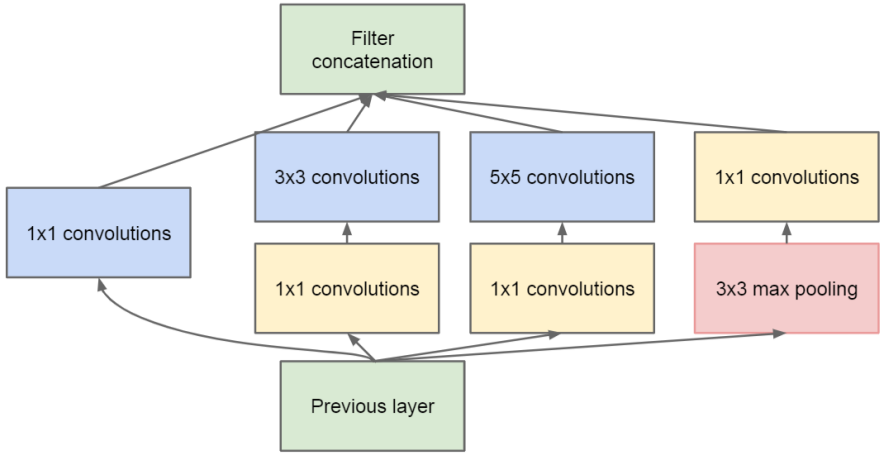
Slide: [Stanford CS231n: Deep Learning for Computer Vision]

GoogLeNet (a.k.a. Inception v1; 2014)

- The 1st place in ImageNet classification 2014
- Network design: **"Deeper network with computational efficiency"**
  - Only 5 million parameters (22 layers)
    - Note) 12x less than AlexNet / 27x less than VGG-16
  - Efficient ***Inception*** module and **no FC layer**
    - Network-in-network with various sizes of CNN kernels
    - 1x1 convolution → computational efficiency
  - Auxiliary classifiers to prevent gradient vanishing



(a) Inception module, naïve version



(b) Inception module with dimension reductions

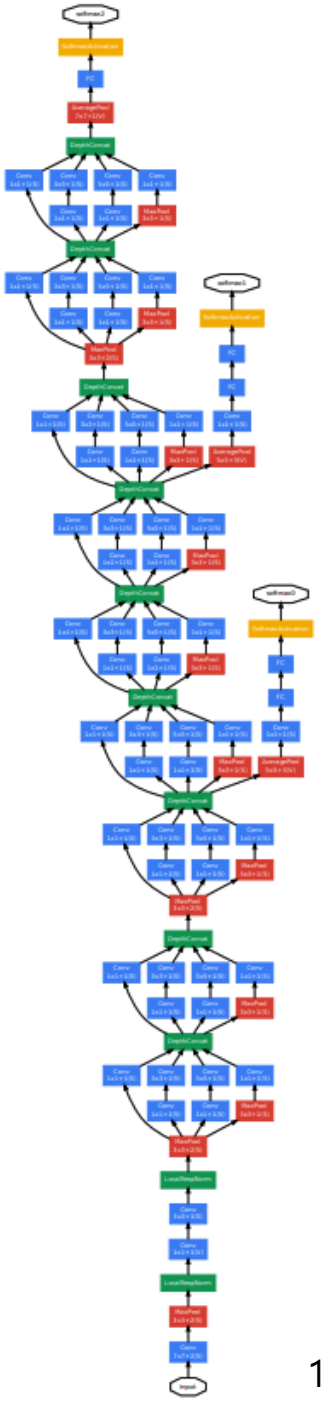


Image: Szegedy et al., **Going Deeper with Convolutions**, CVPR, 2015 DOI arXiv

GoogLeNet (a.k.a. Inception v1; 2014)

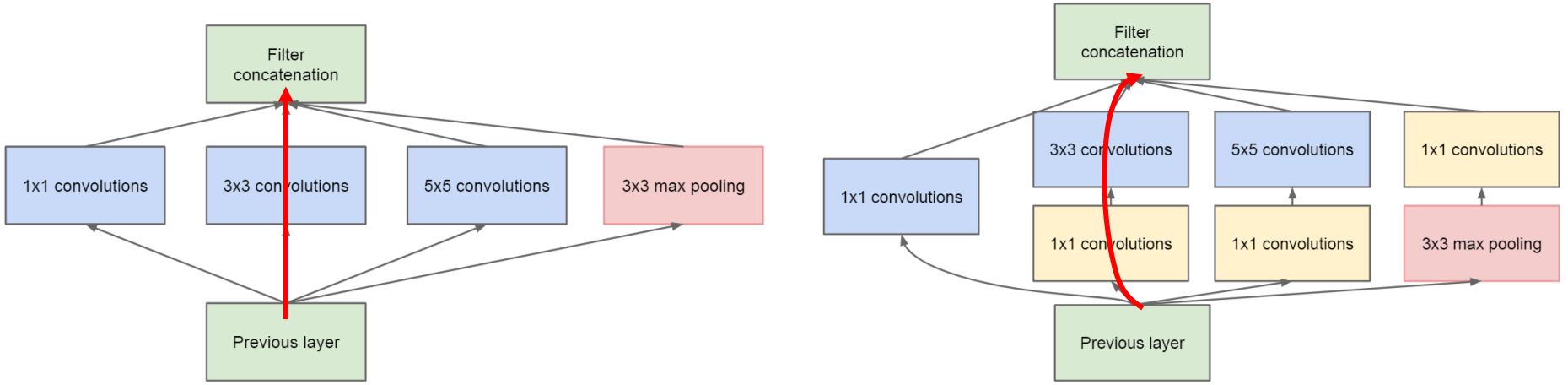
- Network design: **"Deeper network with computational efficiency"**
  - Only 5 million parameters (22 layers)
  - Efficient ***Inception*** module and **no FC layer**
    - 1x1 convolution → computational efficiency
      - e.g. Previous layer: 28x28, 3x3 convolutions

Naïve 3x3 convolution (256 → 128 channels)

- Parameters: 256x128x3x3 = **294,912**
- FLOPs: 3x3x256x128x28x28 ~ **231M**

Bottleneck with 1x1 convolution (256 → **64** → 128 channels)

- Parameters: 256x64x1x1 + 64x128x3x3 = **90,112 (70% ↓)**
- FLOPs: 1x1x256x64x28x28 + 3x3x64x128x28x28 ~ **70.6M (70% ↓)**



(a) Inception module, naïve version (b) Inception module with dimension reductions

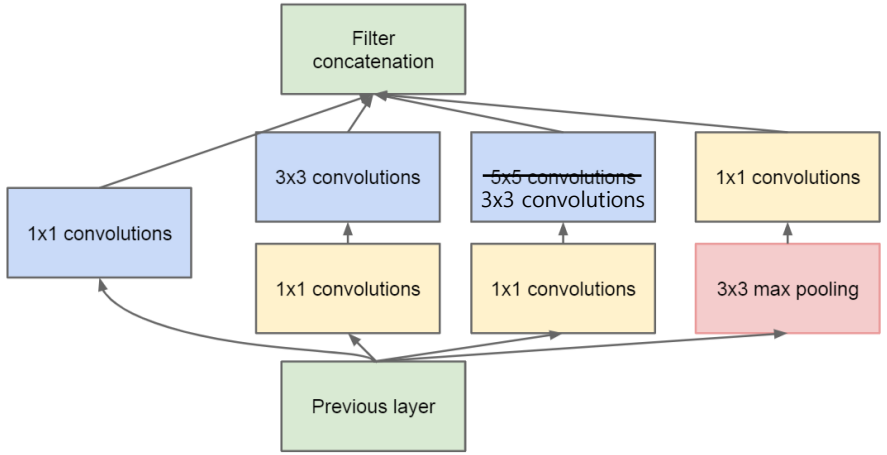
Image: Szegedy et al., **Going Deeper with Convolutions**, CVPR, 2015 DOI arXiv

GoogLeNet (a.k.a. Inception v1; 2014)

```python
import torchvision.models as models

model = models. googlenet(pretrained=False)
print(model)
```

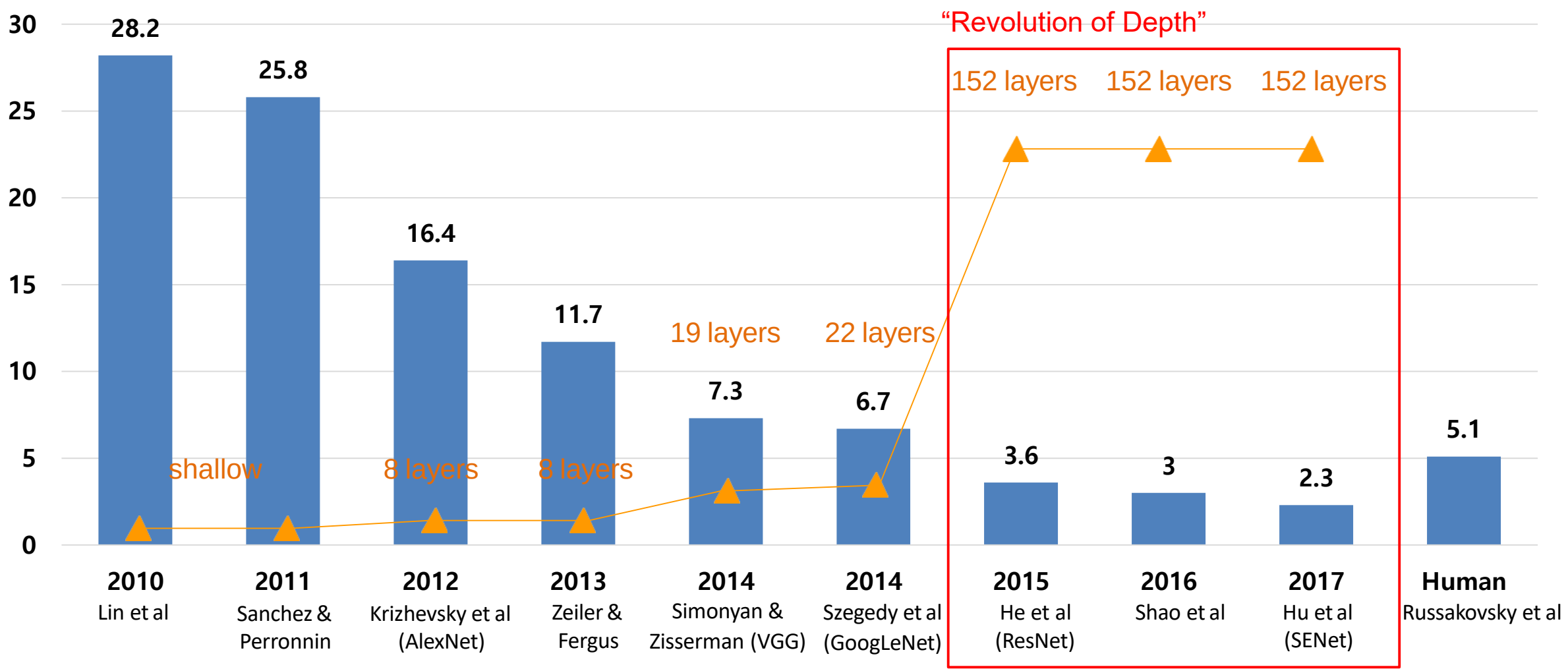
- Example) **GoogLeNet implementation** @ PyTorch (src, doc)

```
GoogLeNet(
  ...
  (inception3a): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(192, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(192, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(96, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(96, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(192, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(16, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1, ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(192, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
```



ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

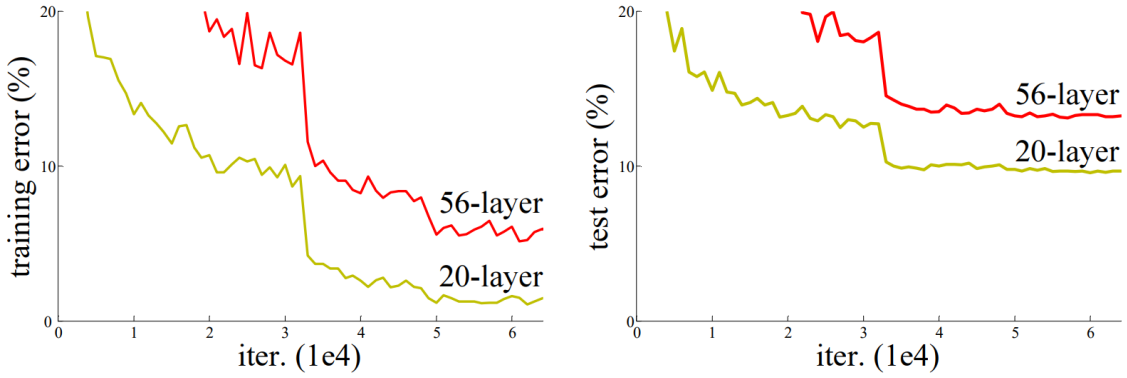
- **Top-5 errors** with their number of NN layers

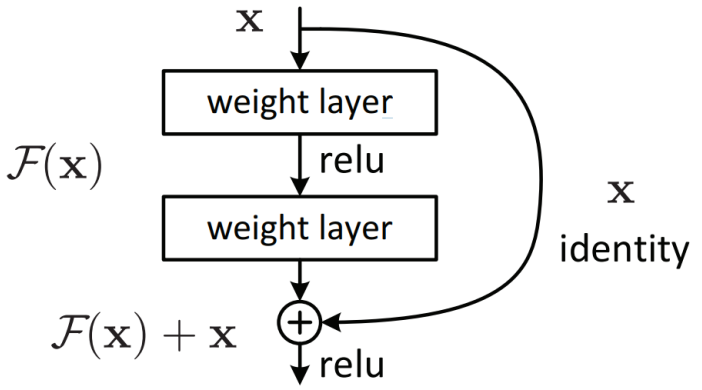


Slide: [Stanford CS231n: Deep Learning for Computer Vision]

ResNet (2015)

- The 1st place in ImageNet classification 2015 (top-5 error: 3.6%)
  - Note) ResNet was the first to outperform human experts (top-5 error: 5.1%).
- Network design: **"Very deep network with residual connections"**
  - Motivation) A deeper network is worse than a swallow network. ← vanishing gradient problem





  - Solution) **Residual connection** (a.k.a. skip connection)
    - A *residual block* directly connects its input layer to its output.
    - The *skip connection* helps the network learn the difference (or residual) rather than full transformation.

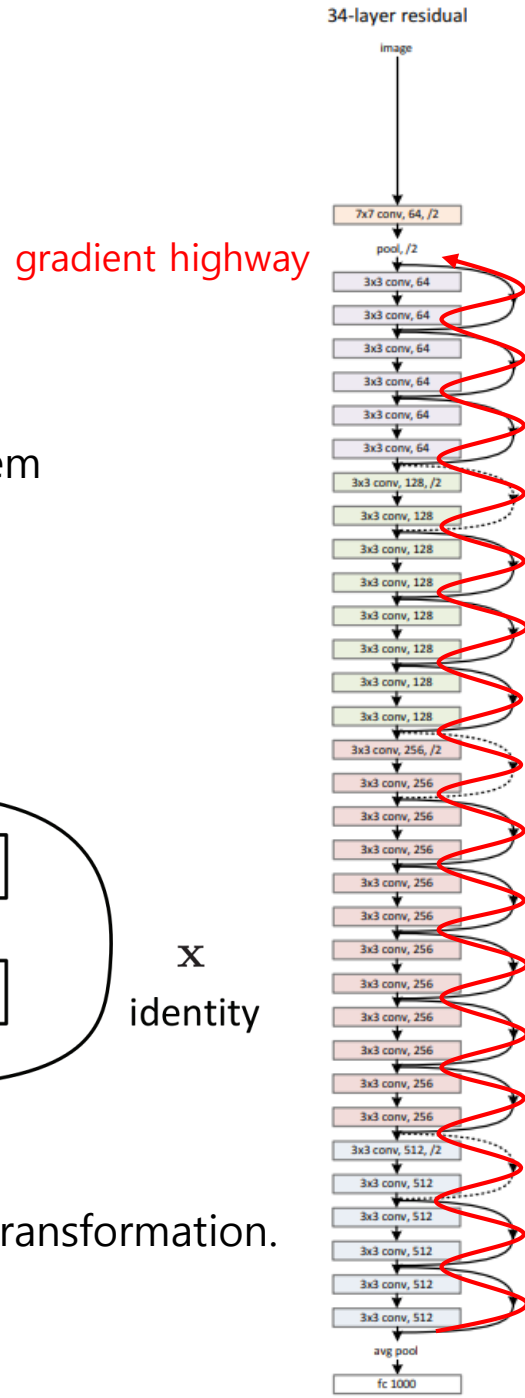
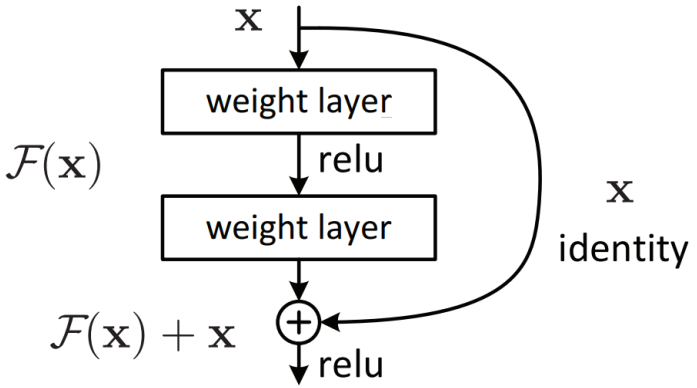
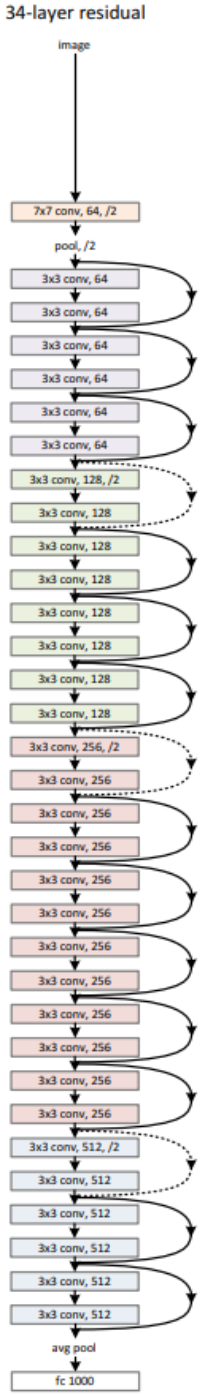


Image: He et al., **Deep Residual Learning for Image Recognition**, CVPR, 2016 CVF arXiv

ResNet (2015)

- Network design: **"Very deep network with residual connections"**
  - **Residual connection** (a.k.a. skip connection)
    - A *residual block* directly connects its input layer to its output.
    - The *skip connection* helps the network learn the difference (or residual) rather than full transformation.
  - **Residual block** with **residual connection**





layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Image: He et al., **Deep Residual Learning for Image Recognition**, CVPR, 2016 CVF arXiv

ResNet (2015)

```python
import torchvision.models as models

model = models.resnet34(pretrained=False)
print(model)
```

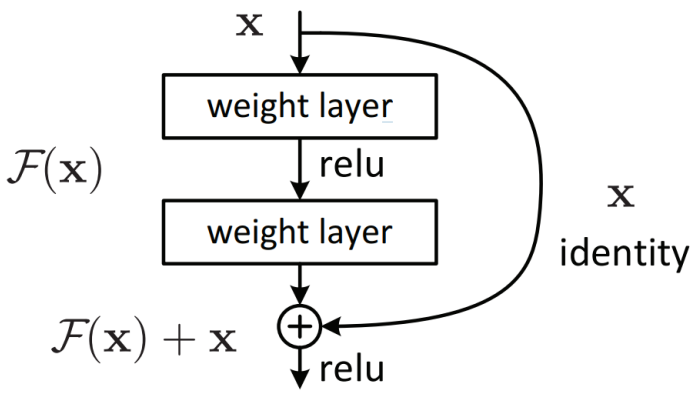
- Example) **ResNet-34 implementation** @ PyTorch (src, doc)

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (2): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

ResNet (2015)

- Example) **ResNet-34 implementation** @ PyTorch (src, doc)

```python
class BasicBlock(nn.Module):
    ...

    def forward(self, x: Tensor) -> Tensor:
        identity = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        if self.downsample is not None:
            identity = self.downsample(x)

        out += identity
        out = self.relu(out)

        return out
```



CNN Backbone Networks

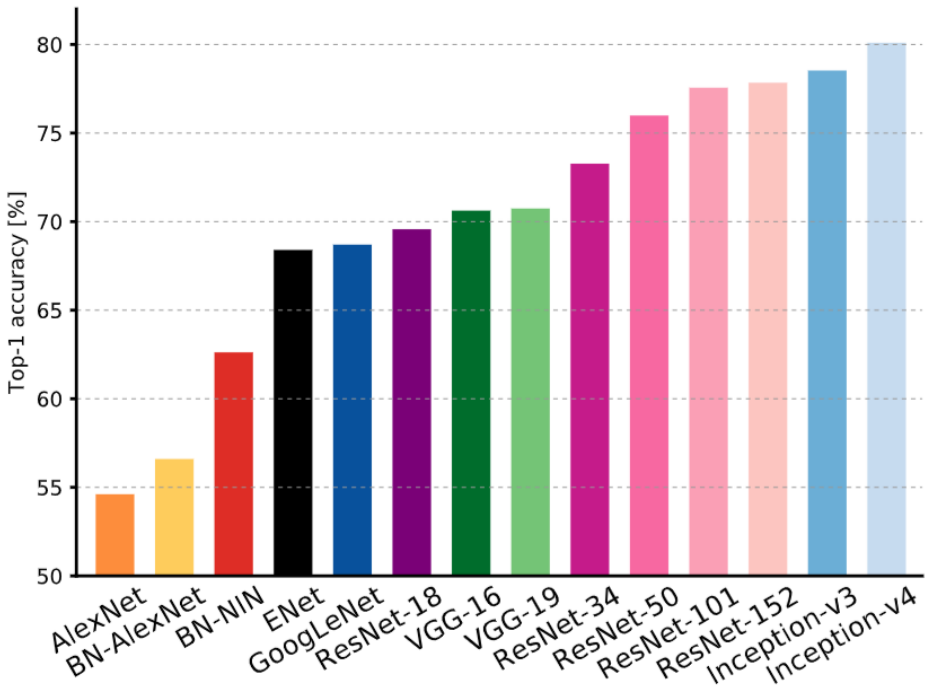


Figure 1: **Top1 *vs.* network.** Single-crop top-1 validation accuracies for top scoring single-model architectures. We introduce with this chart our choice of colour scheme, which will be used throughout this publication to distinguish effectively different architectures and their correspondent authors. Notice that networks of the same group share the same hue, for example ResNet are all variations of pink.

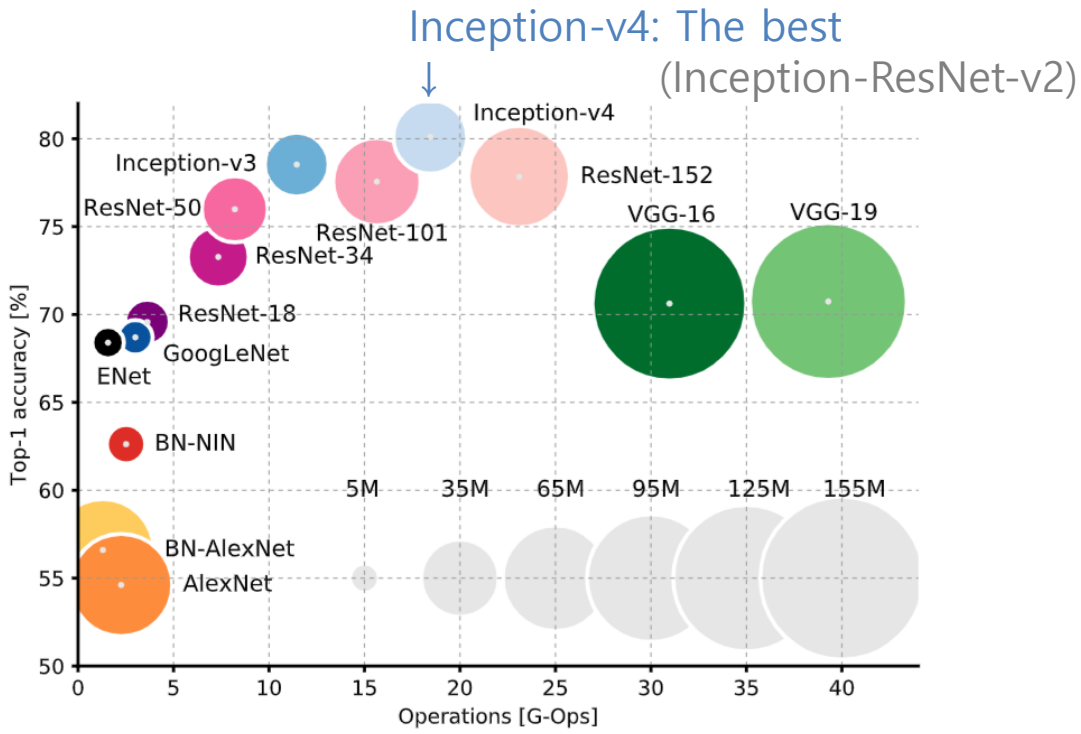


Figure 2: **Top1 *vs.* operations, size $\propto$ parameters.** Top-1 one-crop accuracy versus amount of operations required for a single forward pass. The size of the blobs is proportional to the number of network parameters; a legend is reported in the bottom right corner, spanning from 5×10^6 to 155×10^6 params. Both these figures share the same y-axis, and the grey dots highlight the centre of the blobs.

Image: Canziani et al., **An Analysis of Deep Neural Network Models for Practical Applications**, arXiv, 2016 arXiv

CNN Backbone Networks

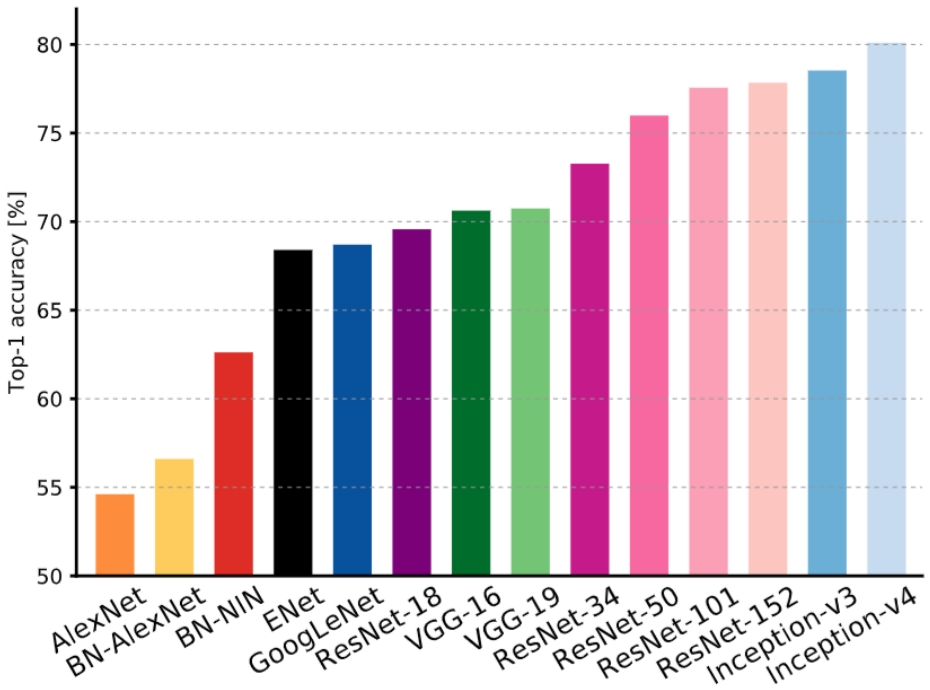


Figure 1: **Top1 *vs.* network.** Single-crop top-1 validation accuracies for top scoring single-model architectures. We introduce with this chart our choice of colour scheme, which will be used throughout this publication to distinguish effectively different architectures and their correspondent authors. Notice that networks of the same group share the same hue, for example ResNet are all variations of pink.

ResNet: Moderate efficiency and high accuracy

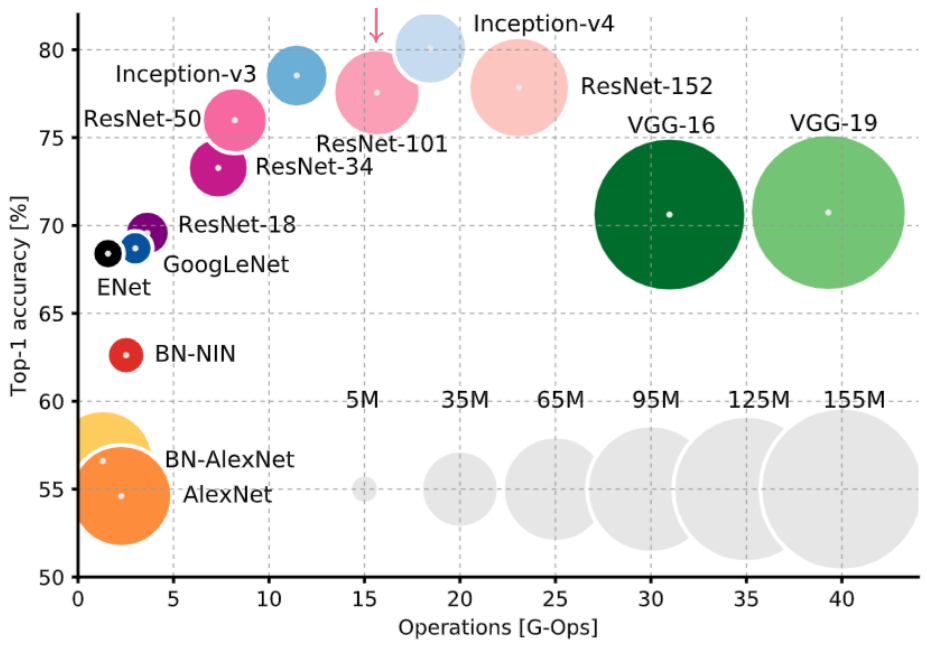


Figure 2: **Top1 *vs.* operations, size $\propto$ parameters.** Top-1 one-crop accuracy versus amount of operations required for a single forward pass. The size of the blobs is proportional to the number of network parameters; a legend is reported in the bottom right corner, spanning from 5×10^6 to 155×10^6 params. Both these figures share the same y-axis, and the grey dots highlight the centre of the blobs.

Image: Canziani et al., **An Analysis of Deep Neural Network Models for Practical Applications**, arXiv, 2016 arXiv

CNN Backbone Networks

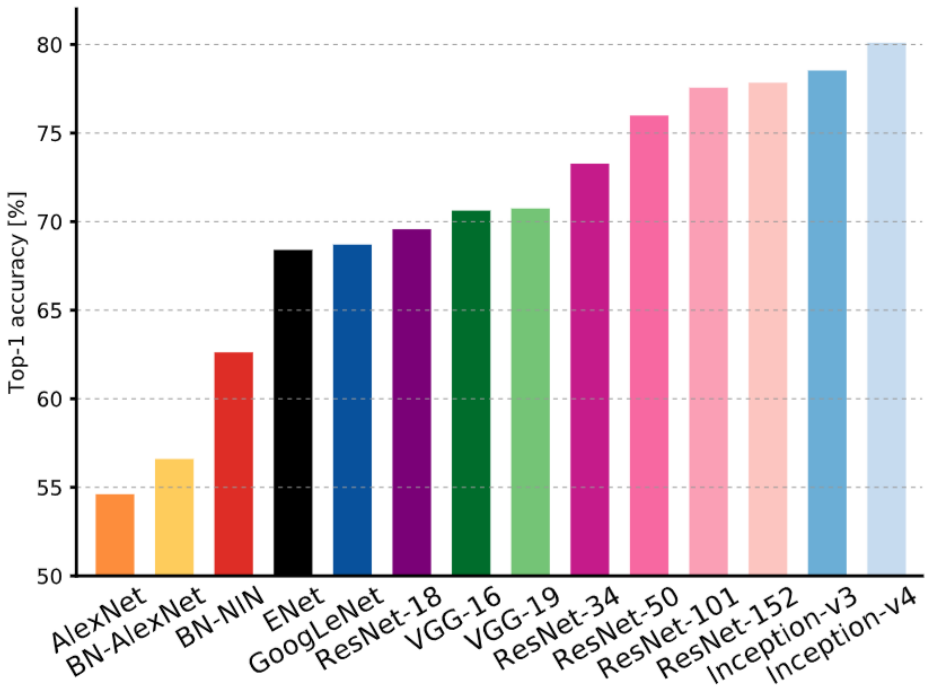


Figure 1: **Top1 *vs.* network.** Single-crop top-1 validation accuracies for top scoring single-model architectures. We introduce with this chart our choice of colour scheme, which will be used throughout this publication to distinguish effectively different architectures and their correspondent authors. Notice that networks of the same group share the same hue, for example ResNet are all variations of pink.

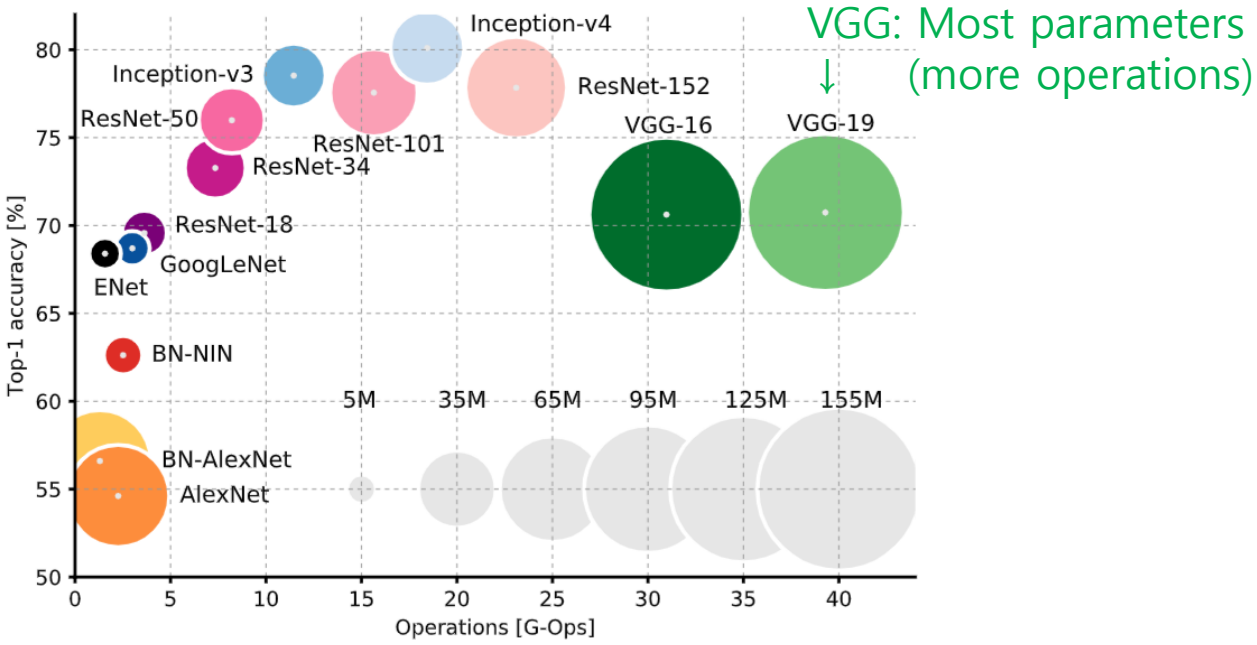


Figure 2: **Top1 *vs.* operations, size $\propto$ parameters.** Top-1 one-crop accuracy versus amount of operations required for a single forward pass. The size of the blobs is proportional to the number of network parameters; a legend is reported in the bottom right corner, spanning from 5×10^6 to 155×10^6 params. Both these figures share the same y-axis, and the grey dots highlight the centre of the blobs.

Image: Canziani et al., **An Analysis of Deep Neural Network Models for Practical Applications**, arXiv, 2016 arXiv

CNN Backbone Networks

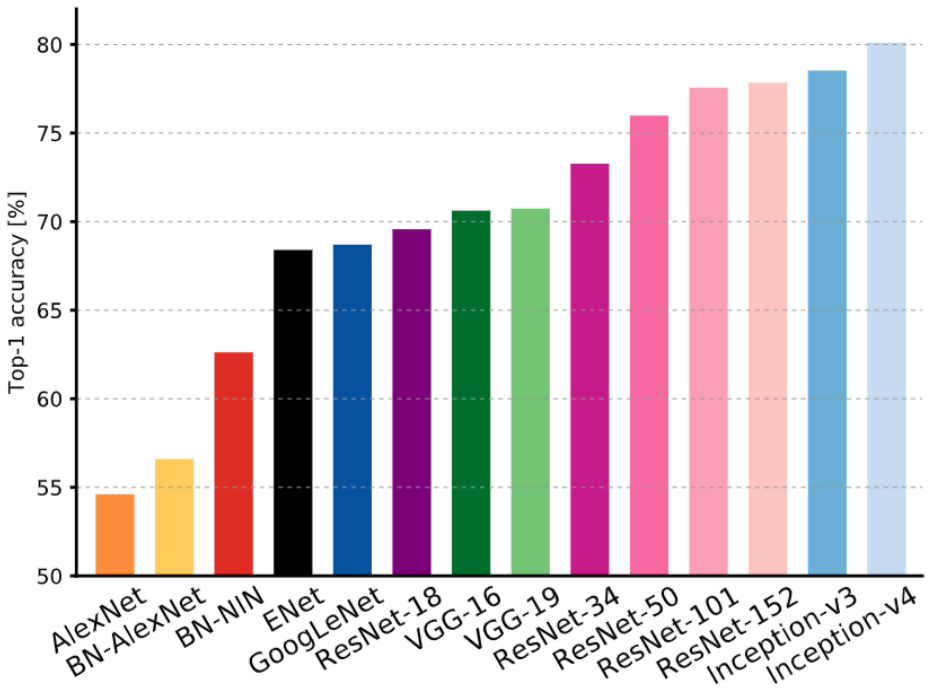


Figure 1: **Top1 *vs.* network.** Single-crop top-1 validation accuracies for top scoring single-model architectures. We introduce with this chart our choice of colour scheme, which will be used throughout this publication to distinguish effectively different architectures and their correspondent authors. Notice that networks of the same group share the same hue, for example ResNet are all variations of pink.

GoogLeNet: The most efficient

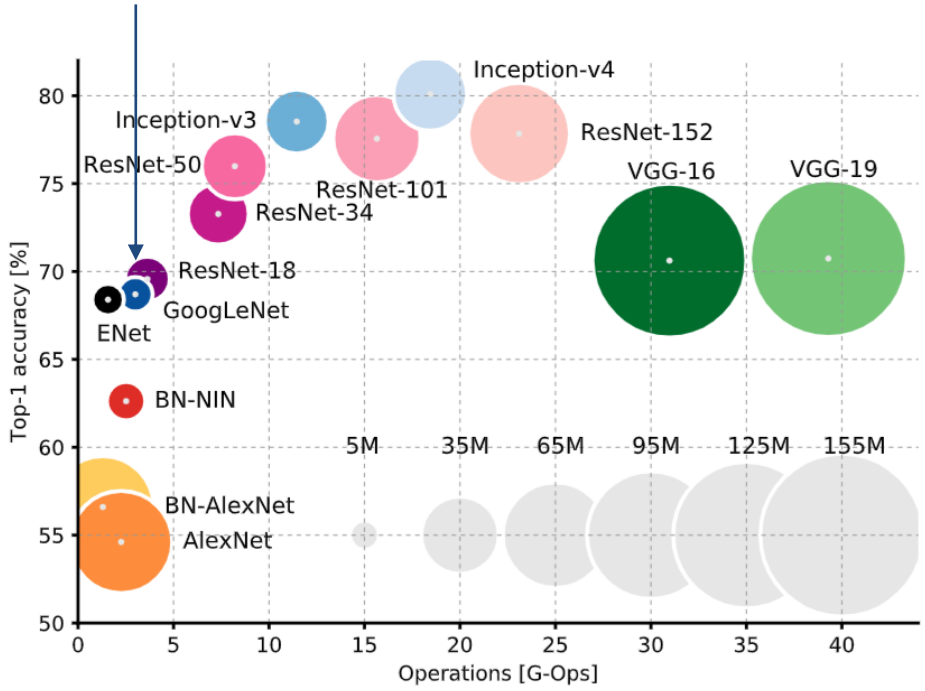


Figure 2: **Top1 *vs.* operations, size $\propto$ parameters.** Top-1 one-crop accuracy versus amount of operations required for a single forward pass. The size of the blobs is proportional to the number of network parameters; a legend is reported in the bottom right corner, spanning from 5×10^6 to 155×10^6 params. Both these figures share the same y-axis, and the grey dots highlight the centre of the blobs.

Image: Canziani et al., **An Analysis of Deep Neural Network Models for Practical Applications**, arXiv, 2016 arXiv

CNN Backbone Networks

AlexNet: Smaller operations, More parameters, Lower accuracy

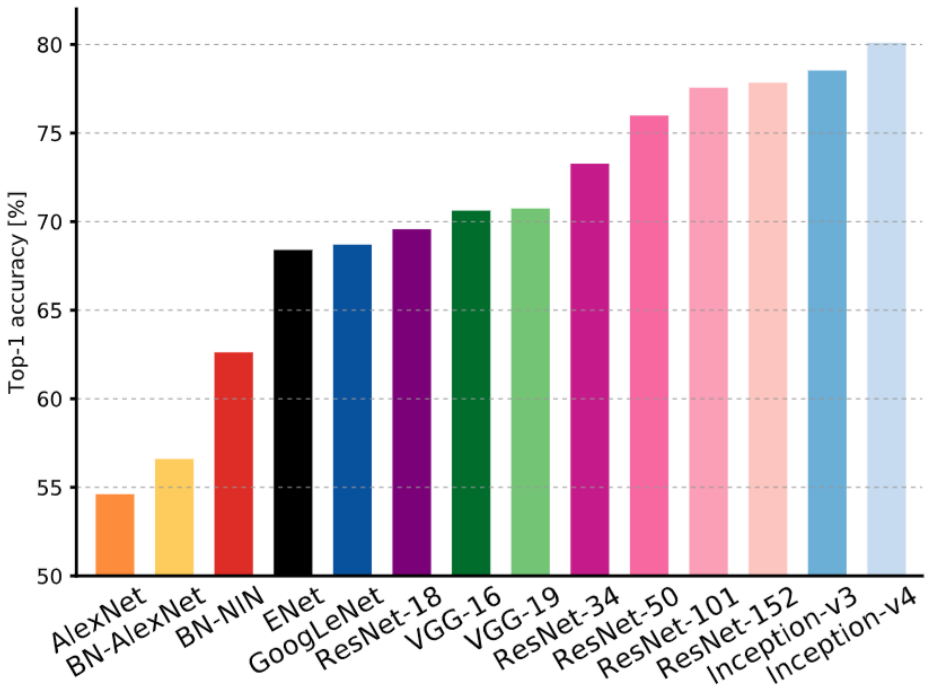


Figure 1: **Top1 *vs.* network.** Single-crop top-1 validation accuracies for top scoring single-model architectures. We introduce with this chart our choice of colour scheme, which will be used throughout this publication to distinguish effectively different architectures and their correspondent authors. Notice that networks of the same group share the same hue, for example ResNet are all variations of pink.

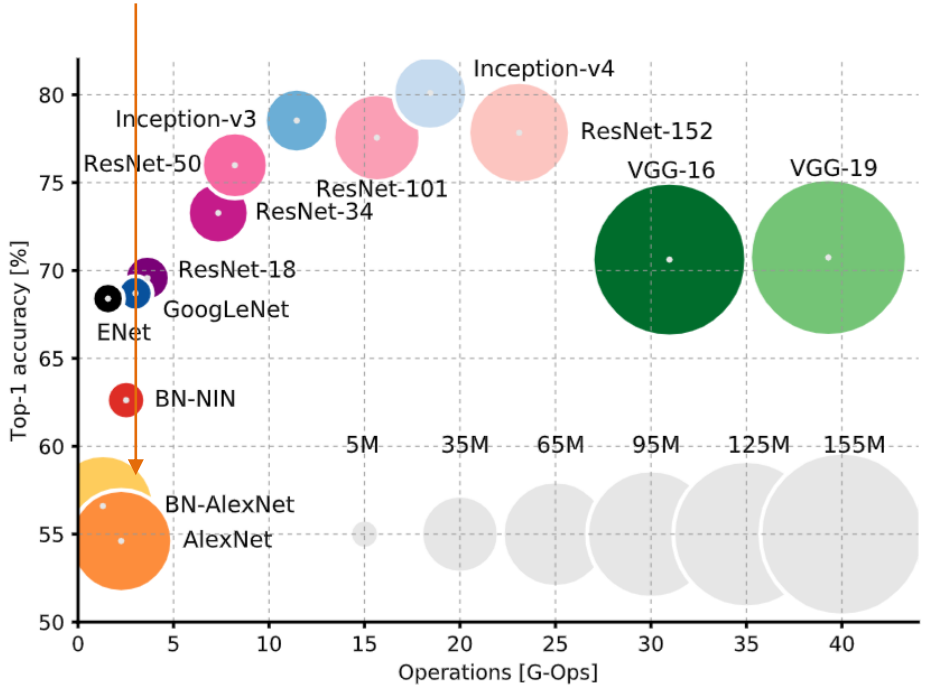
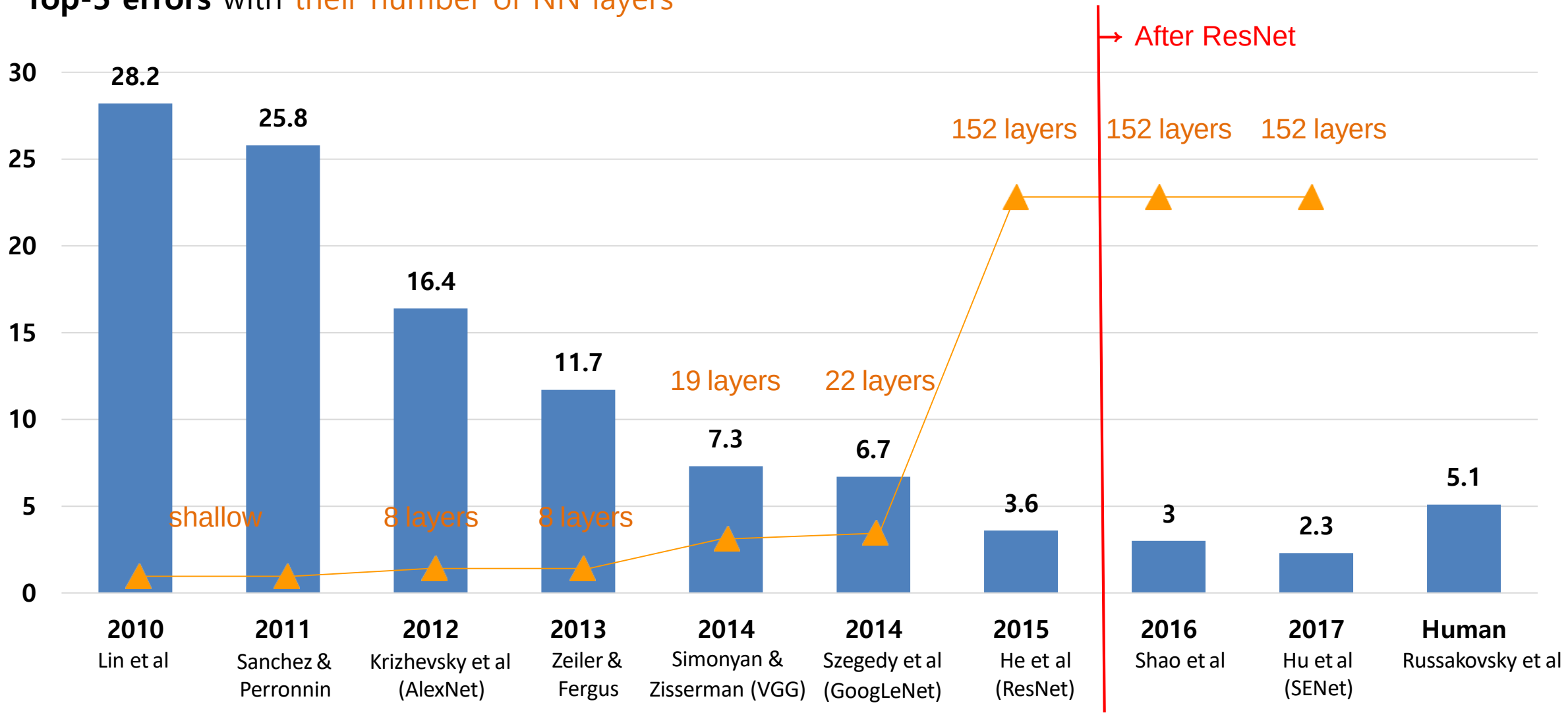


Figure 2: **Top1 *vs.* operations, size $\propto$ parameters.** Top-1 one-crop accuracy versus amount of operations required for a single forward pass. The size of the blobs is proportional to the number of network parameters; a legend is reported in the bottom right corner, spanning from 5×10^6 to 155×10^6 params. Both these figures share the same y-axis, and the grey dots highlight the centre of the blobs.

Image: Canziani et al., **An Analysis of Deep Neural Network Models for Practical Applications**, arXiv, 2016 arXiv

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

- **Top-5 errors** with their number of NN layers



Slide: [Stanford CS231n: Deep Learning for Computer Vision]

SENet: Squeeze-and-Excitation Networks (2017)

- Key idea: Considering channel-wise weights (~ attention mechanism)

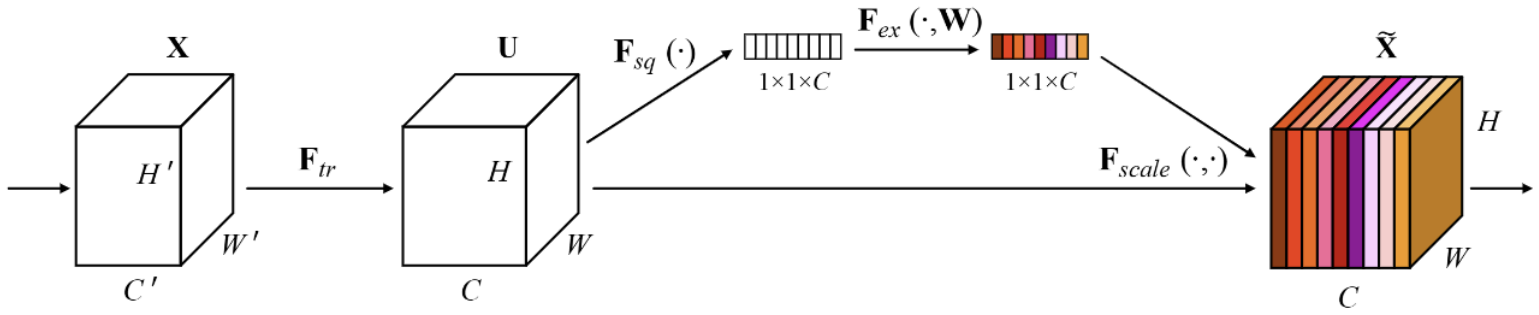


Figure 1: A Squeeze-and-Excitation block.

	224 × 224		320 × 320 / 299 × 299	
	top-1 err.	top-5 err.	top-1 err.	top-5 err.
ResNet-152 [10]	23.0	6.7	21.3	5.5
ResNet-200 [11]	21.7	5.8	20.1	4.8
Inception-v3 [44]	-	-	21.2	5.6
Inception-v4 [42]	-	-	20.0	5.0
Inception-ResNet-v2 [42]	-	-	19.9	4.9
ResNeXt-101 (64 × 4d) [47]	20.4	5.3	19.1	4.4
DenseNet-264 [14]	22.15	6.12	-	-
Attention-92 [46]	-	-	19.5	4.8
Very Deep PolyNet [51] †	-	-	18.71	4.25
PyramidNet-200 [8]	20.1	5.4	19.2	4.7
DPN-131 [5]	19.93	5.12	18.55	4.16
SENet-154	**18.68**	**4.47**	**17.28**	**3.79**
NASNet-A (6@4032) [55] †	-	-	17.3‡	3.8‡
SENet-154 (post-challenge)	-	-	**16.88‡**	**3.58‡**

Table 4: Single-crop error rates of state-of-the-art CNNs on ImageNet validation set. The size of test crop is 224 × 224 and 320 × 320 / 299 × 299 as in [11]. † denotes the model with a larger crop 331 × 331. ‡ denotes the post-challenge result. SENet-154 (post-challenge) is trained with a larger input size 320 × 320 compared to the original one with the input size 224 × 224.

Image: Hu et al., **Squeeze-and-Excitation Networks**, CVPR, 2018 CVF Github (authors) Github

ResNeXt (2016)

- Key idea: Small but grouped (parallel) convolutions (~ inception modules)

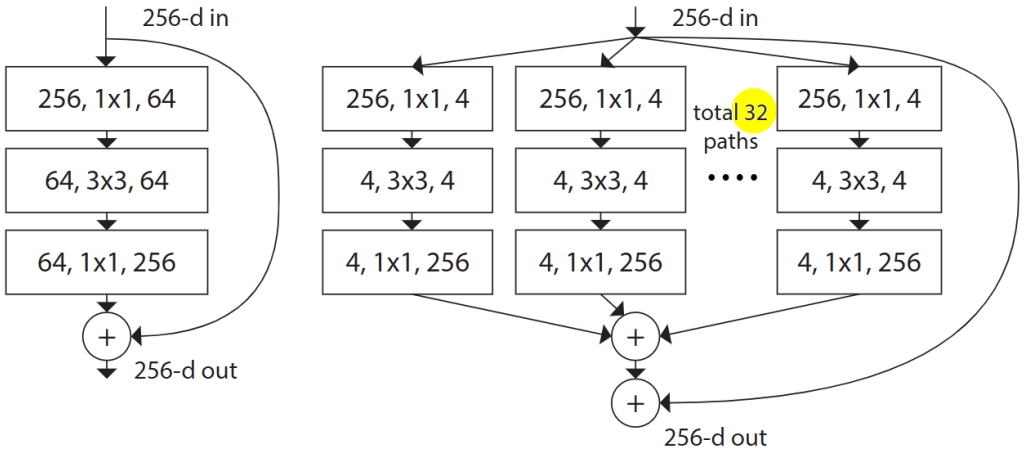


Figure 1. **Left**: A block of ResNet [14]. **Right**: A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

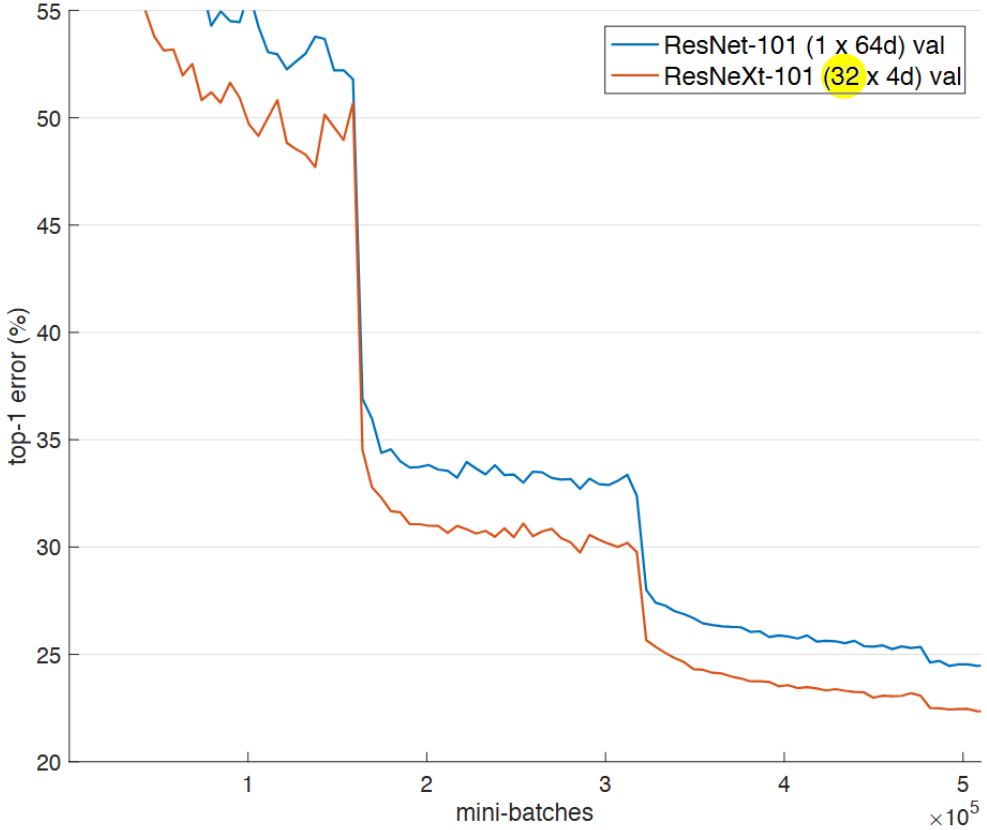


Figure 6. **ImageNet-5K** experiments. Models are trained on the 5K set and evaluated on the original 1K validation set, plotted as a 1K-way classification task. ResNeXt and its ResNet counterpart have similar complexity.

Image: Xie et al., **Aggregated Residual Transformations for Deep Neural Network**, CVPR, 2017 CVF

DenseNet (2016)

- Key idea: Concatenating (not addition) previous feature maps

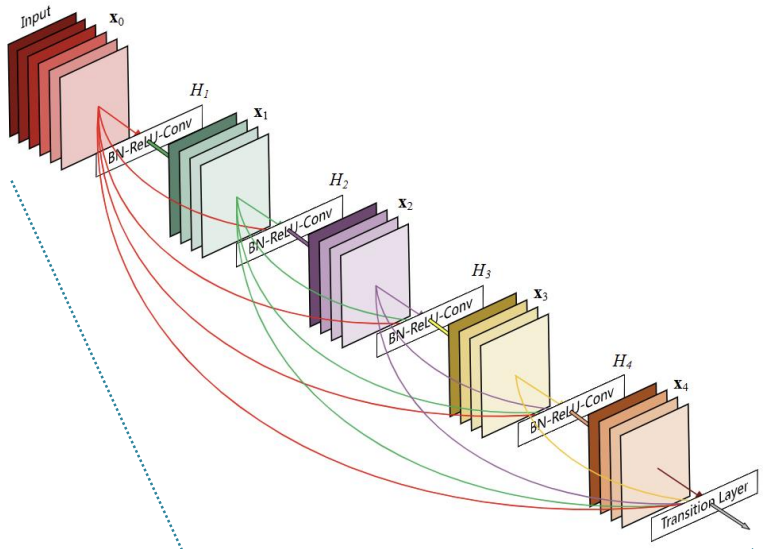


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

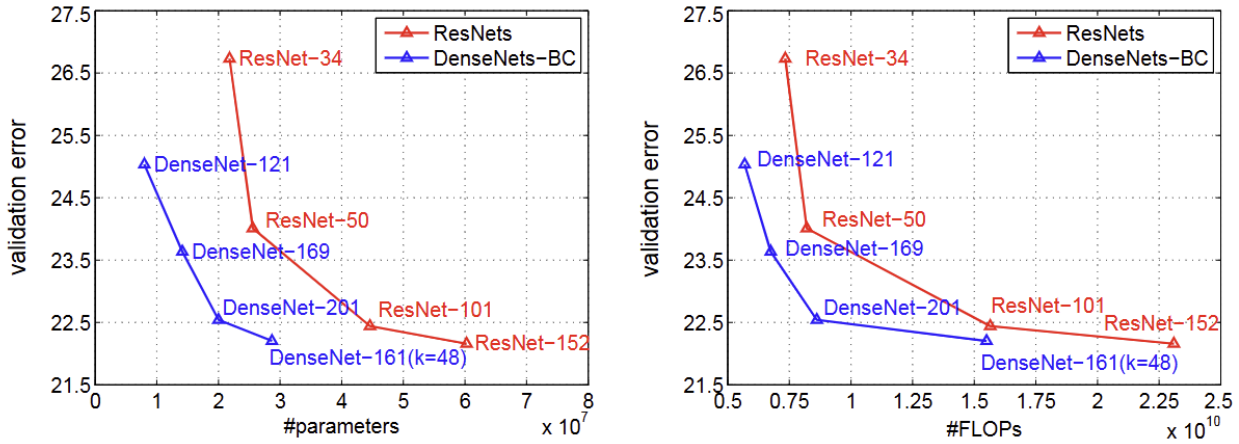


Figure 3: Comparison of the DenseNets and ResNets top-1 error rates (single-crop testing) on the ImageNet validation dataset as a function of learned parameters (*left*) and FLOPs during test-time (*right*).

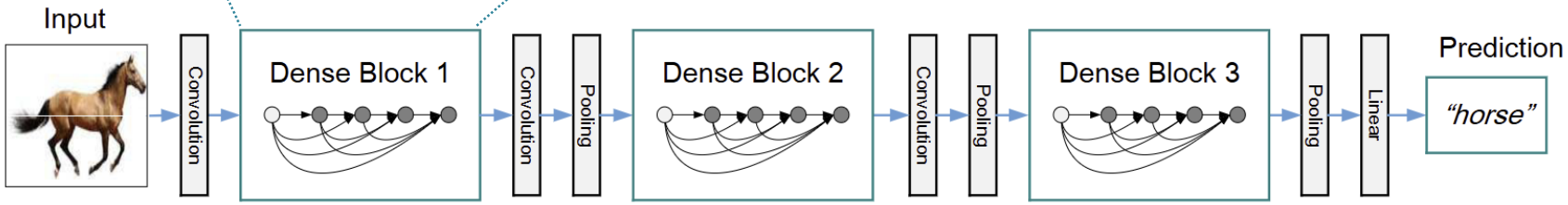


Figure 2: A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

Image: Huang et al., **Densely Connected Convolutional Networks**, CVPR, 2017 CVF

MobileNet (2017)

- Key idea: Decompose a convolution as *depthwise* convolutions and a *pointwise* convolution

  → Less parameters and computation with slightly less accuracy

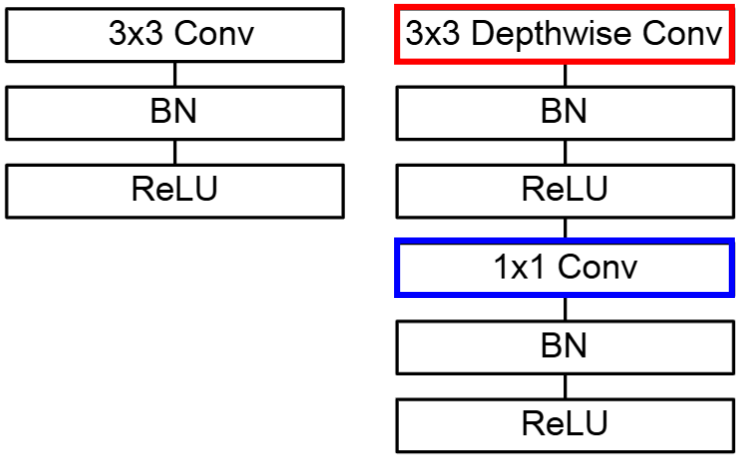


Figure 3. Left: Standard convolutional layer with batchnorm and ReLU. Right: Depthwise Separable convolutions with Depthwise and Pointwise layers followed by batchnorm and ReLU.

Table 8. MobileNet Comparison to Popular Models

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogleNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

Table 9. Smaller MobileNet Comparison to Popular Models

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
0.50 MobileNet-160	60.2%	76	1.32
Squeezenet	57.5%	1700	1.25
AlexNet	57.2%	720	60

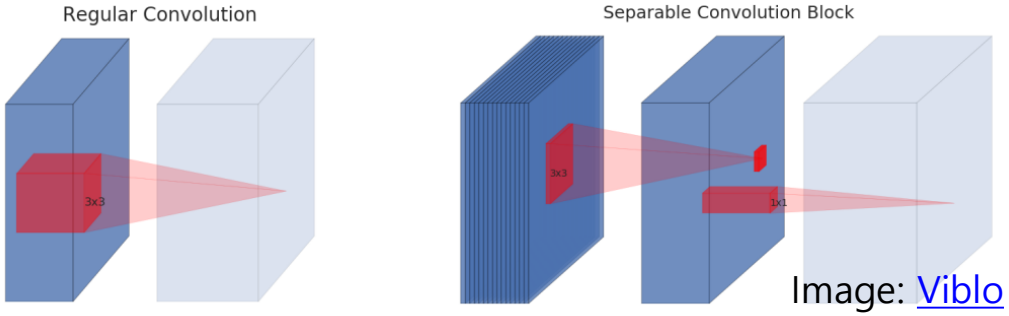


Image: Viblo

Image: Howard et al., **MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications**, arXiv, 2017 arXiv

EfficientNet (2019)

- Key idea: Automation of finding optimal network architectures (neural architecture search; NAS)

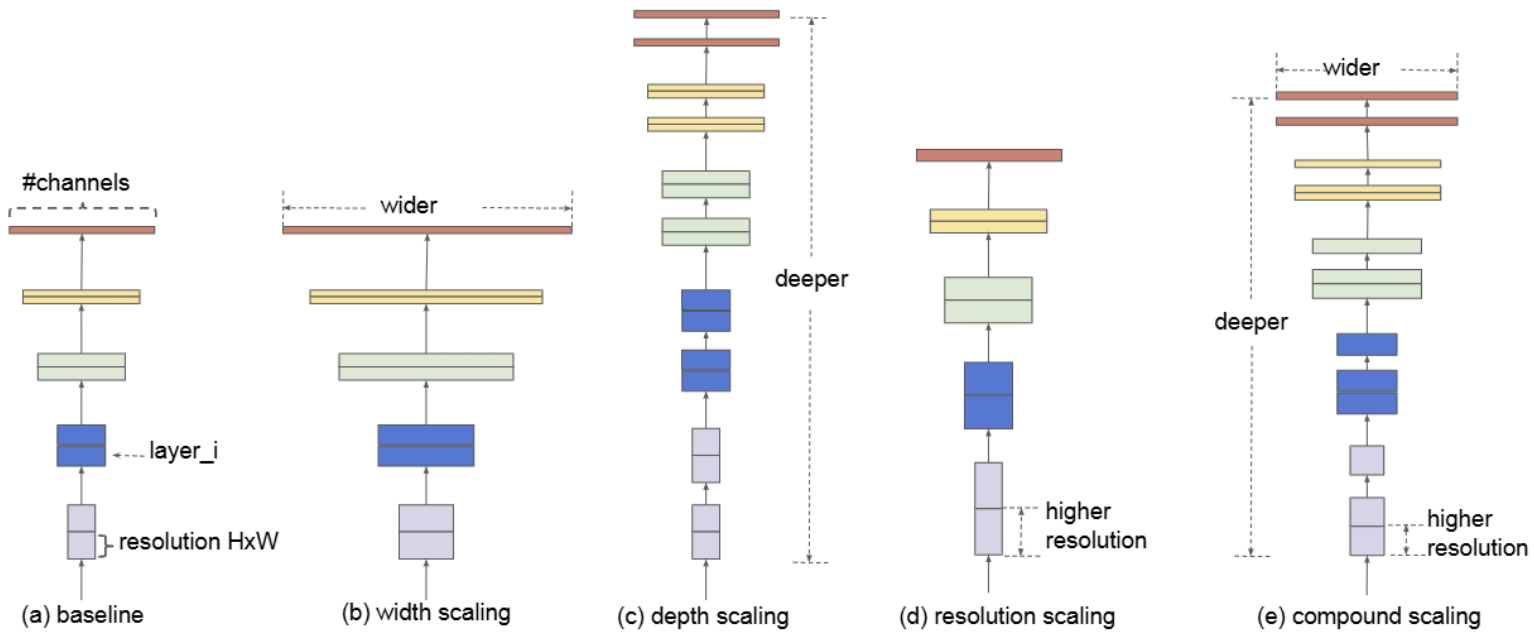


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

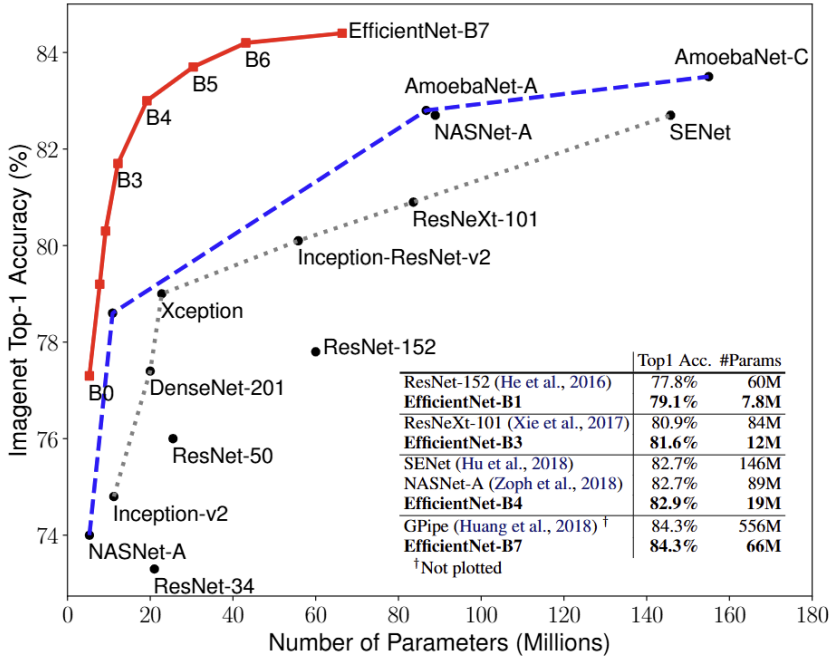
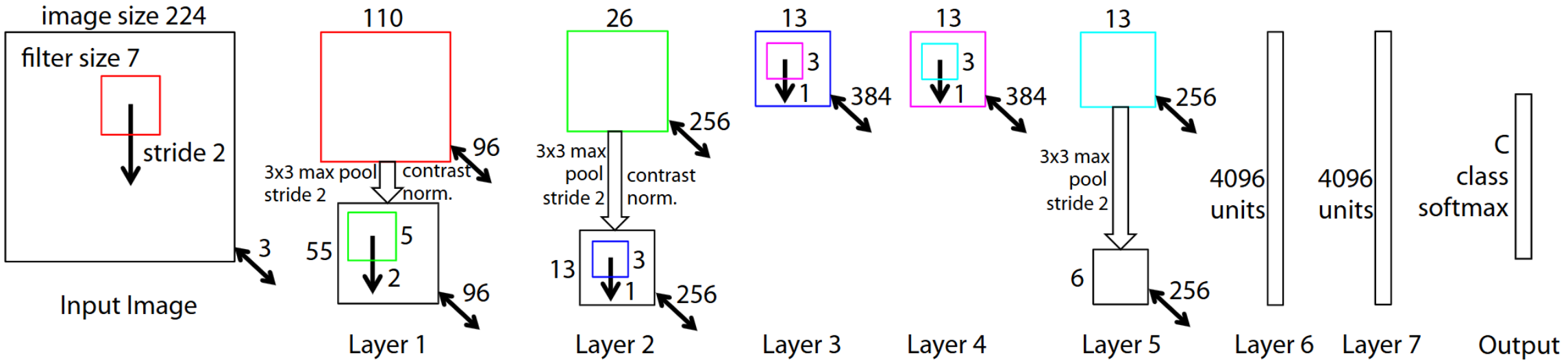


Figure 1. **Model Size vs. ImageNet Accuracy.** All numbers are for single-crop, single-model. Our EfficientNets significantly outperform other ConvNets. In particular, EfficientNet-B7 achieves new state-of-the-art 84.3% top-1 accuracy but being 8.4x smaller and 6.1x faster than GPipe. EfficientNet-B1 is 7.6x smaller and 5.7x faster than ResNet-152. Details are in Table 2 and 4.

Image: Huang et al., **EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks**, ICML, 2019 CVF

Understanding Convolutional Networks

- e.g. Feature examples of AlexNet (2012)



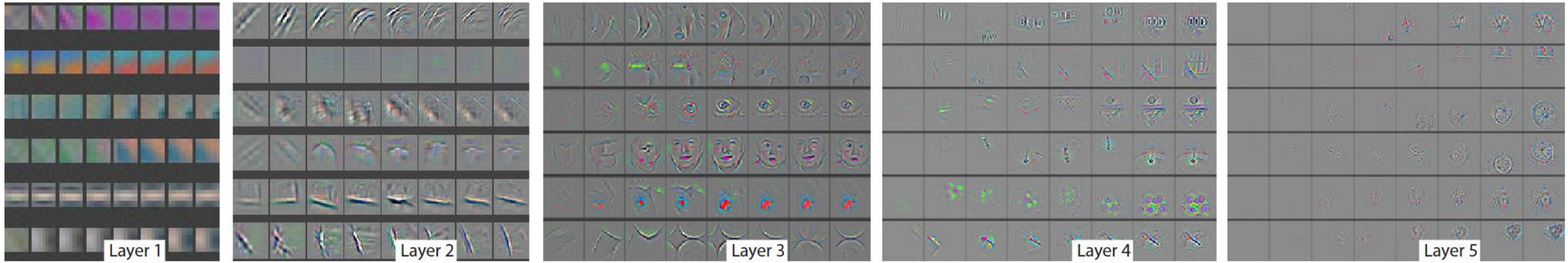
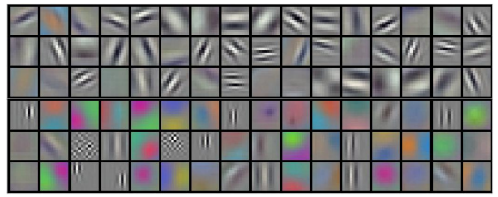
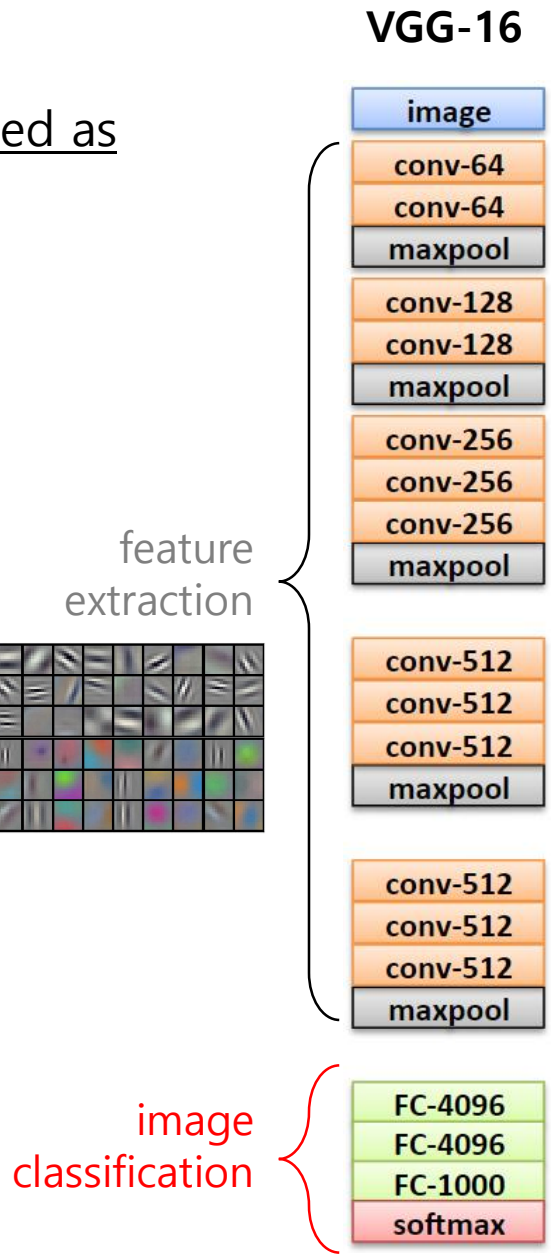


Image: Zeiler et al., **Visualizing and Understanding Convolutional Networks**, ECCV, 2014 arXiv

Transfer Learning

- **Transfer learning** is a technique where a model developed for *one* task is reused as the starting point for a model for an *another* task.
  - Especially useful in deep learning *when data or computation is limited*.
  - Why? Training CNNs from scratch is time-consuming and requires large datasets.
  - Common strategies
    - **Feature Extraction**
      - Use a **pretrained CNN** as a feature extractor
      - Remove final classification layers
    - **Fine-Tuning**
      - Add new last layers for an another task
        - Unfreeze some of the **pretrained model** if necessary
      - Train the network with a lower learning rate

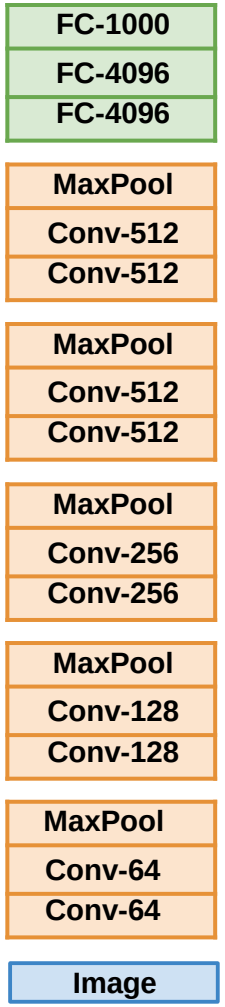




Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014 Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

1. Train on Imagenet

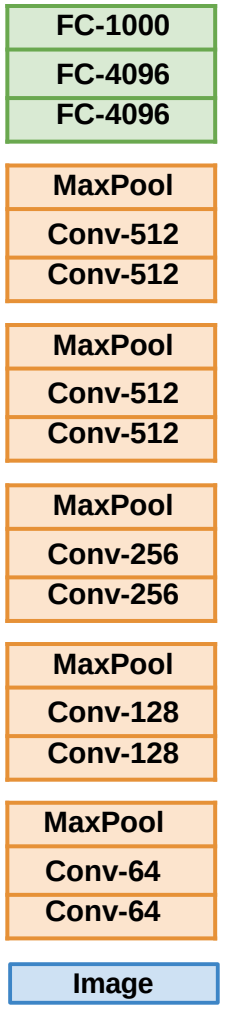


Slide: [Stanford CS231n: Deep Learning for Computer Vision]

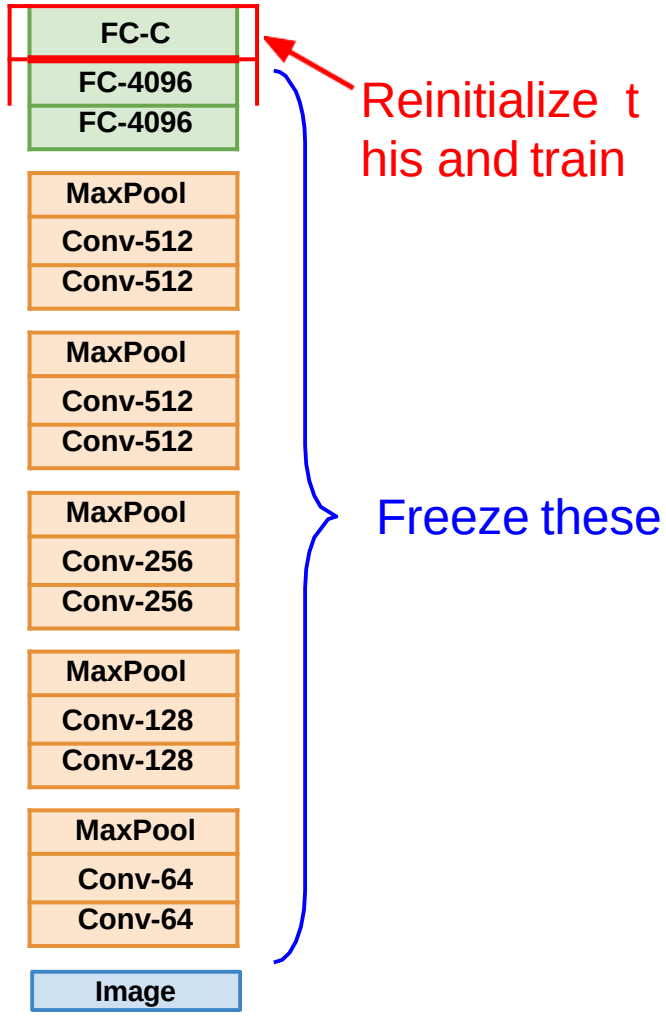
Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014 Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

1. Train on Imagenet



2. Small Dataset (C classes)

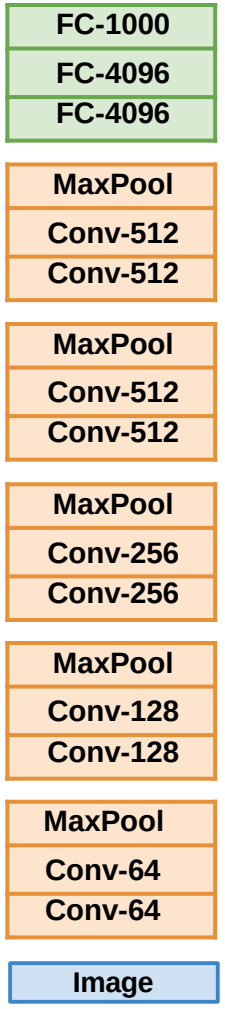


Slide: [Stanford CS231n: Deep Learning for Computer Vision]

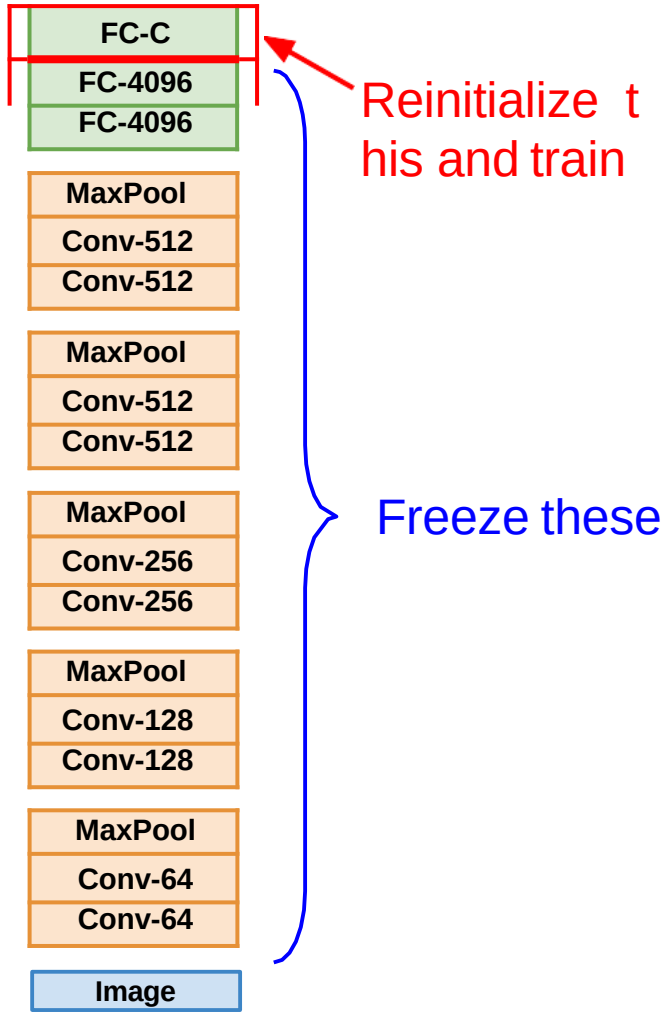
Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014 Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

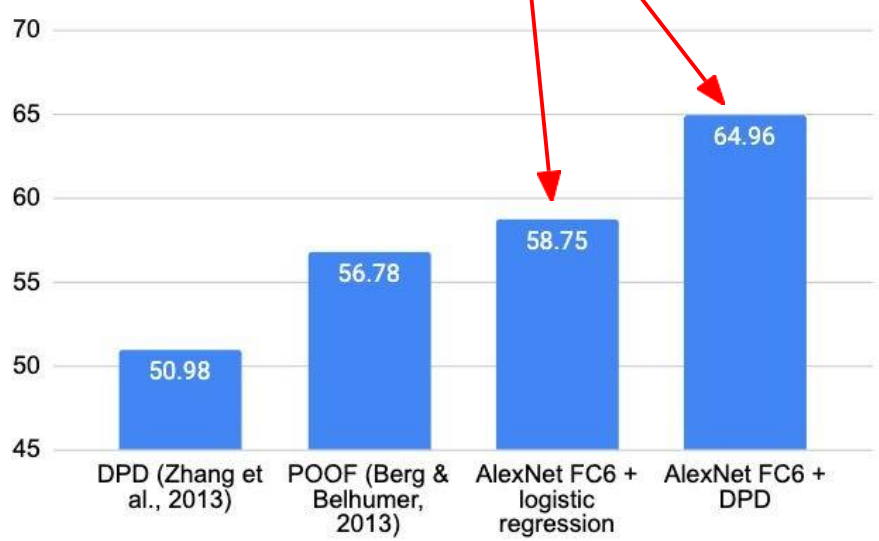
1. Train on Imagenet



2. Small Dataset (C classes)



Finetuned from AlexNet

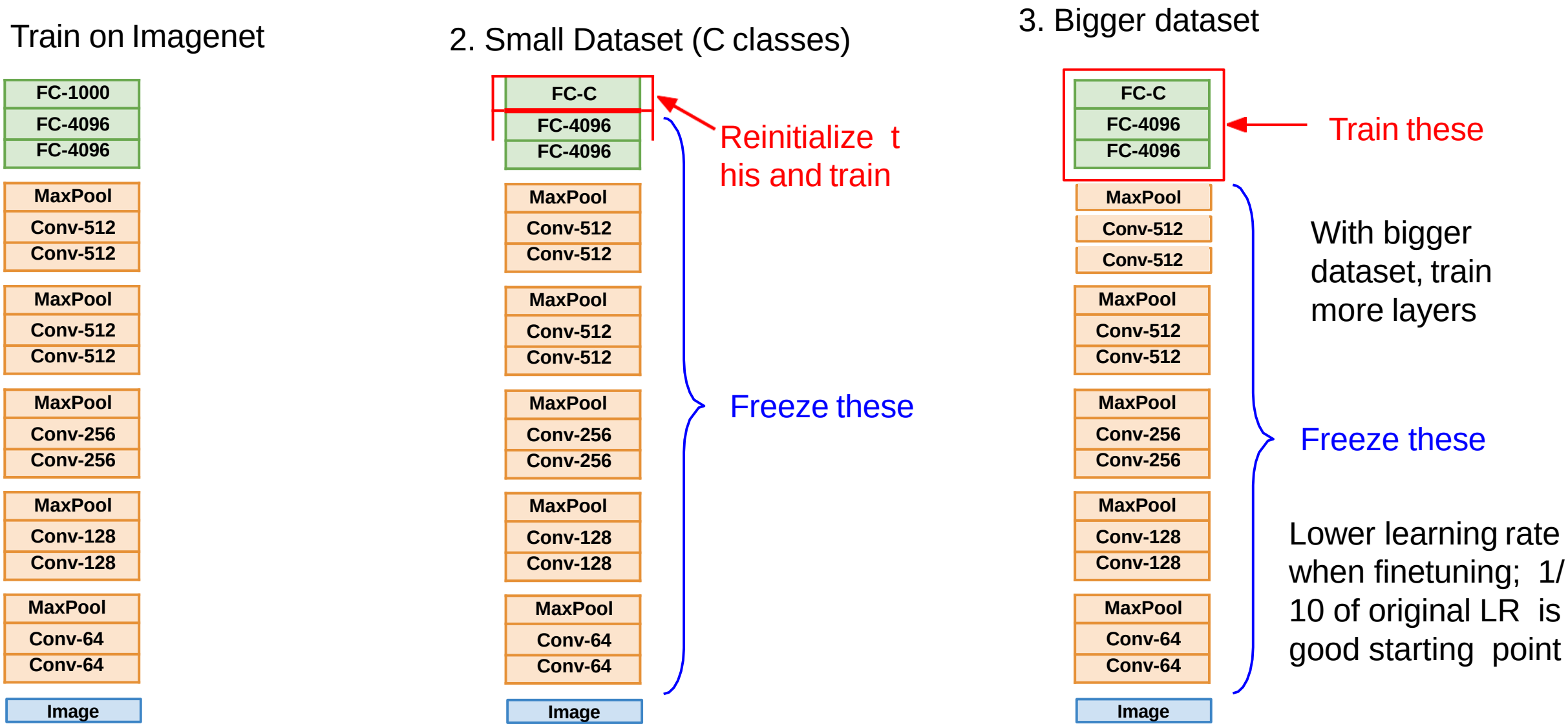


Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014

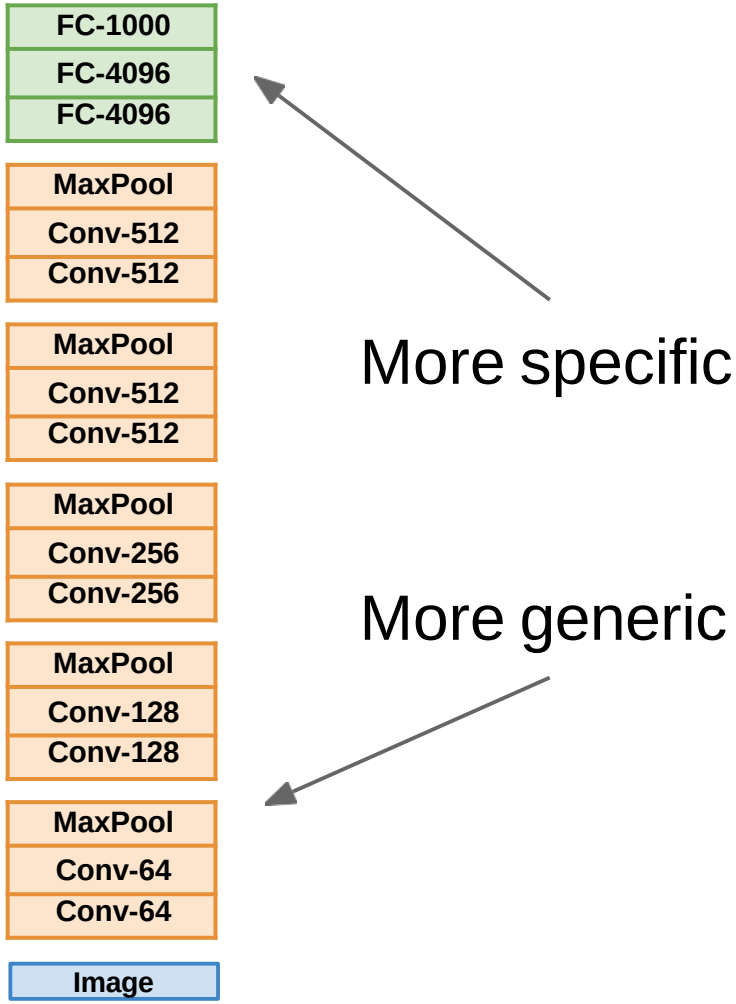
Slide: Stanford CS231n: Deep Learning for Computer Vision

Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014 Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014



Slide: Stanford CS231n: Deep Learning for Computer Vision



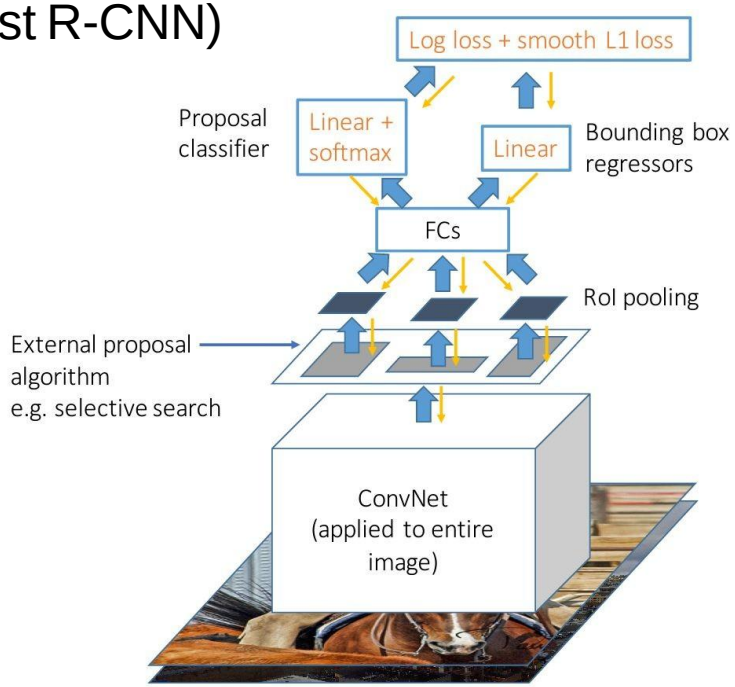
	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Transfer learning with CNNs is pervasive…

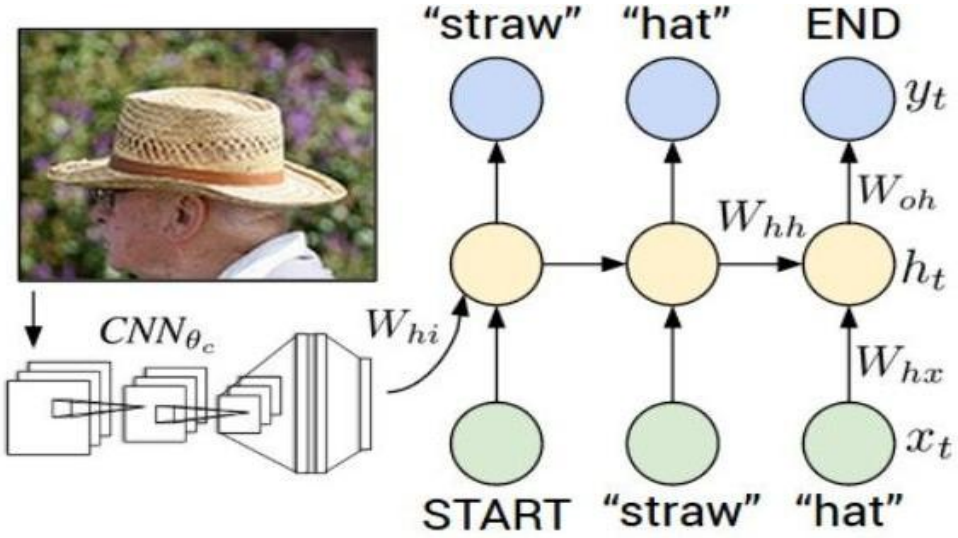
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Image Captioning: CNN + RNN



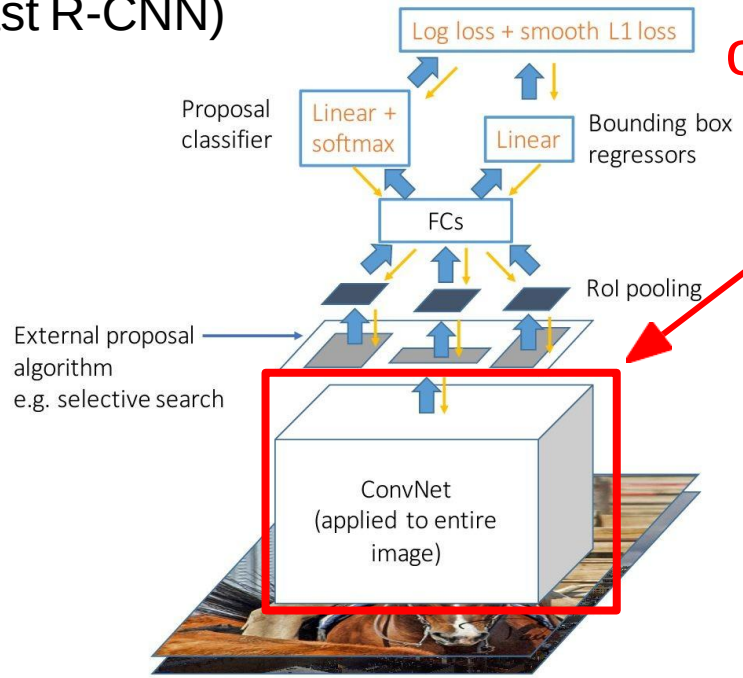
Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Transfer learning with CNNs is pervasive…

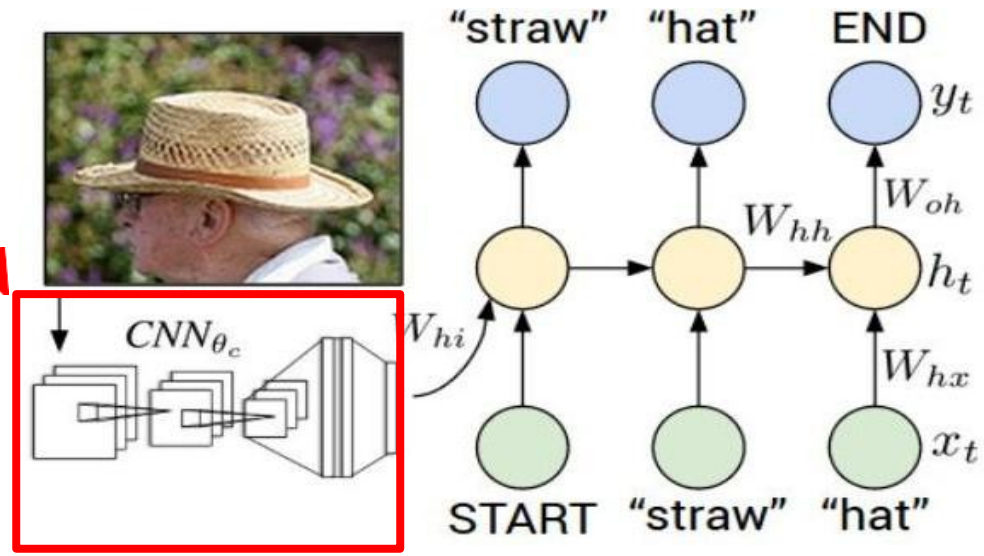
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained on ImageNet

Image Captioning: CNN + RNN



Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

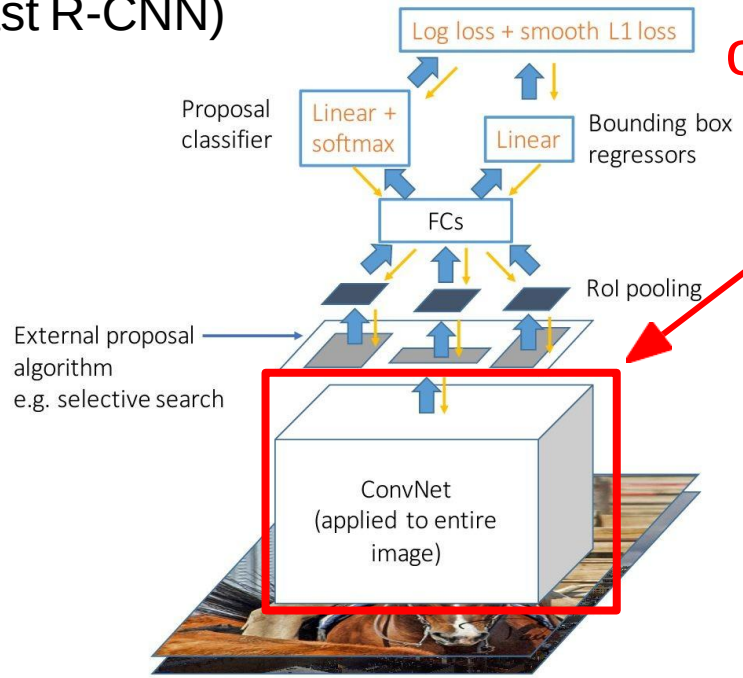
Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Slide: Stanford CS231n: Deep Learning for Computer Vision

Transfer learning with CNNs is pervasive…

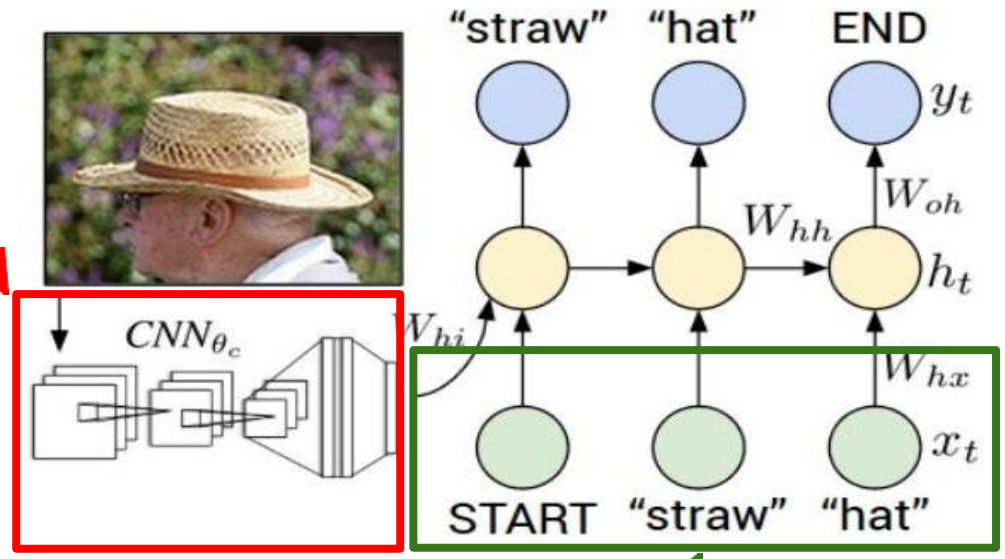
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained on ImageNet

Image Captioning: CNN + RNN



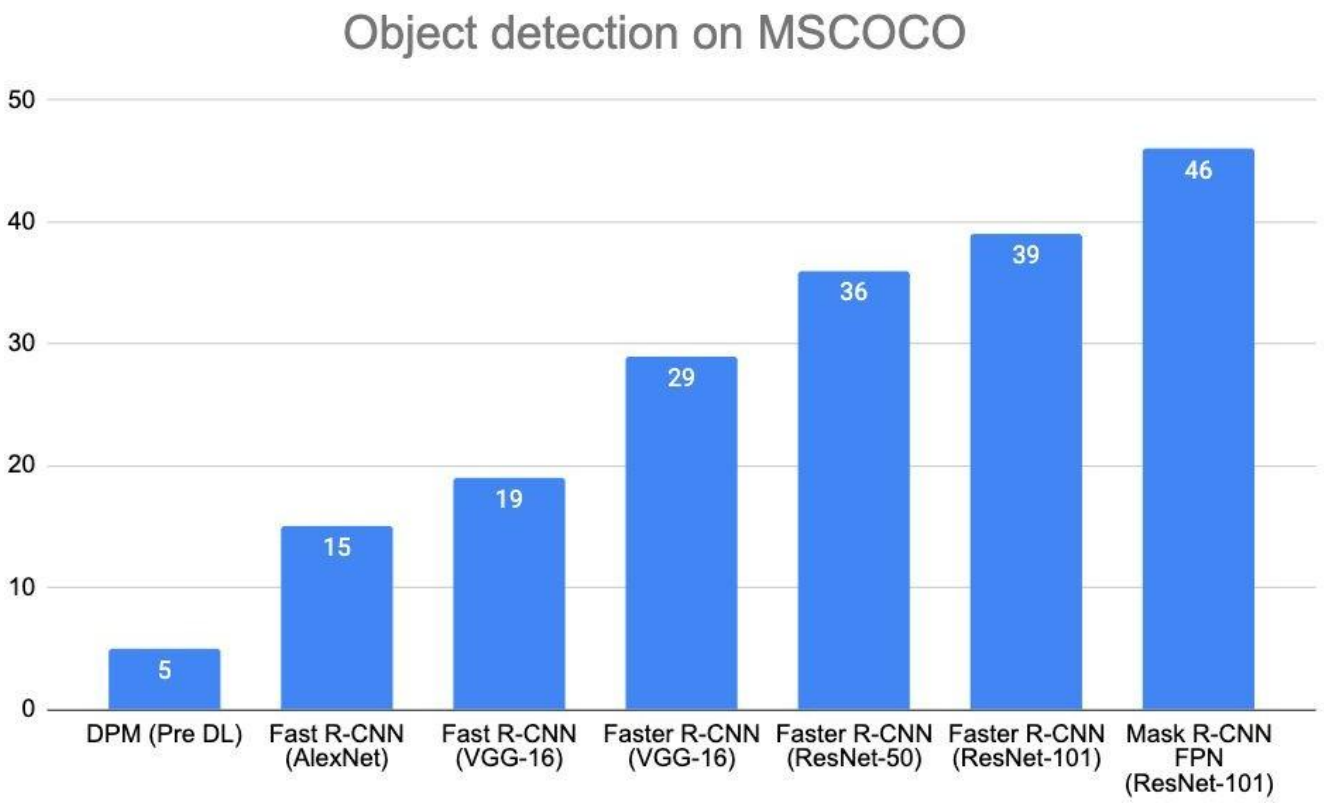
Word vectors pretrained with word2vec

Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Slide: Stanford CS231n: Deep Learning for Computer Vision

Transfer learning with CNNs - Architecture matters

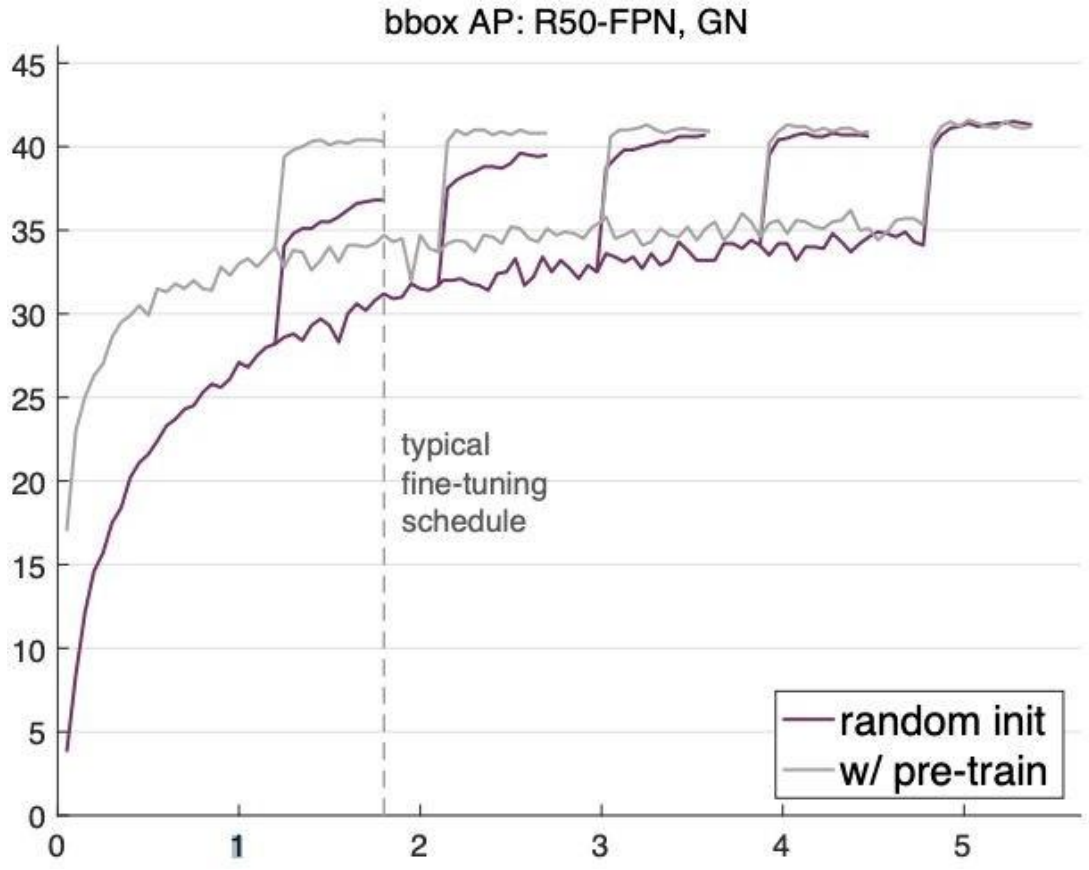


Girshick, "The Generalized R-CNN Framework for Object Detection", ICCV 2017 Tutorial on Instance-Level Visual Recognition

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Transfer learning with CNNs is pervasive…

But recent results show it might not always be necessary!



He et al, “Rethinking ImageNet Pre-training”, ICCV 2019
Figure copyright Kaiming He, 2019. Reproduced with permission.

Training from scratch can work just as well as training from a pretrained ImageNet model for object detection

But it takes 2-3x as long to train.

They also find that collecting more data is better than finetuning on a related task

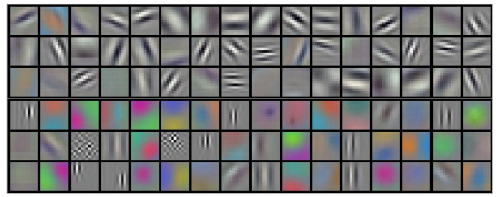
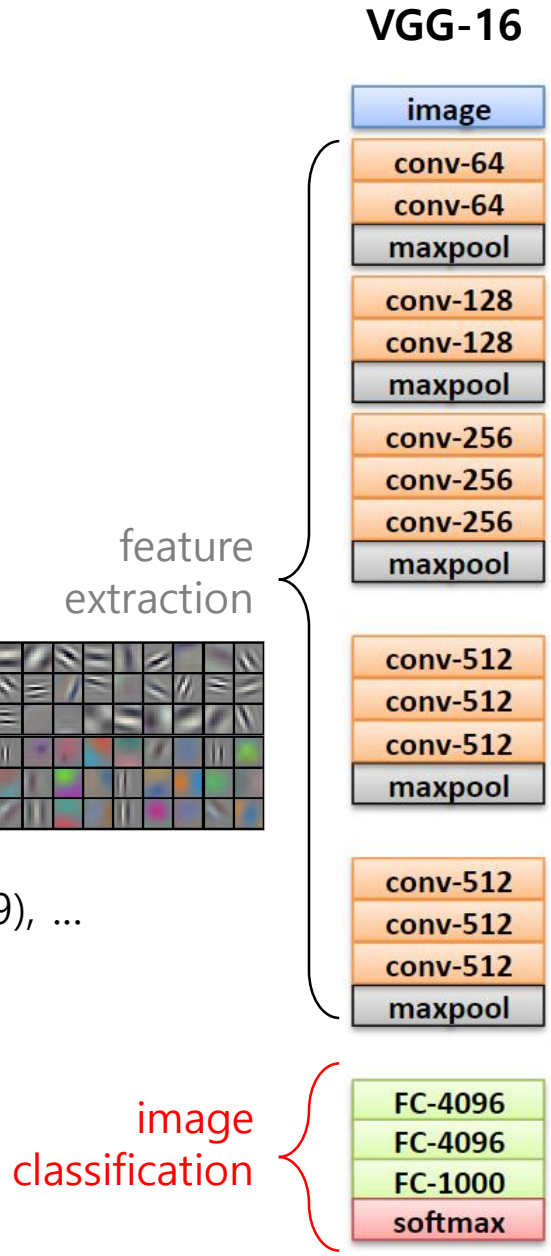
Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Summary

- Image classification: $\text{class} = f(\text{image})$
  - ImageNet Challenge (2010-2017)

- CNN backbone networks
  - LeNet-5 (1998): The first CNN network
  - **AlexNet** (2012): The first CNN-based winner in ImageNet classification
  - **VGGNet** (2014): Small (only **3x3 convolution**) filter but deeper network
  - **GoogLeNet** (2014): Deeper network with computational efficiency
    - **Inception modules with 1x1 convolutions**
  - **ResNet** (2015): Very deep network with **residual connections**
  - SENet (2017), ResNeXt (2016), DenseNet (2016), MobileNet (2017), EfficientNet (2019), …
    - [Convolutional Neural Networks @ Paper With Code]

deeper



- Why are CNN backbone networks important?
  - **Transfer learning** beyond image classification